Dossier de Projet

Application de Gestion de dépôt de documents des intervenants

(APP GDI)

20/04/2026 au 12/06/2026



Préparation au titre de Développeur·se Web et Web Mobile

Yasmine HAILOUL

Remerciements

Je tiens à exprimer ma sincère gratitude à l'ensemble de l'équipe informatique d'ASKORIA pour leur accueil chaleureux, leur disponibilité et leurs conseils tout au long de mon stage.

Je remercie particulièrement la responsable de développement web, Lucie Touchard pour son accompagnement, sa patience et la confiance qu'elle m'a accordé.

Je suis également très reconnaissante envers mon tuteur de stage, Patrick Coupé, pour son suivi attentif, ses retours et ses encouragements qui ont fortement contribué à la réussite de cette expérience professionnelle.

Enfin, un grand merci à toute l'équipe pour leur bonne humeur et l'ambiance de travail agréable qu'ils m'ont fait partager.

Présentation du titre DWWM

Le titre professionnel Développeur Web et Web Mobile, ou DWWM, est une certification de niveau 5 (Bac+2) reconnue par l'État. Il atteste des compétences nécessaires à l'exercice du métier de développeur web, couvrant à la fois la partie visible des applications, appelée front-end, et la partie serveur et bases de données, appelée back-end.

La certification est structurée en deux blocs de compétences, appelés CCP, qui peuvent être obtenus séparément. Le premier bloc concerne le développement front-end sécurisé : il inclut la maquettisation d'interfaces web et mobiles, la réalisation d'interfaces statiques et dynamiques, ainsi que l'installation et la configuration de l'environnement de travail. Le second bloc est dédié au back-end : il couvre la mise en place de bases de données relationnelles, le développement de composants d'accès aux données SQL et NoSQL, le développement de composants métier côté serveur, et la documentation du déploiement des applications.

Dossier de Projet 1
Remerciements 1
Présentation du titre 2
Introduction 4
Découverte de l'établissement ASKORIA et de l'équipe informatique 5
Mise en place de l'environnement de développement 6
Outils, langages, systèmes 7
Analyse et conception de l'application 8
Modélisation des processus métier avec les diagrammes d'activité 9
Spécification des besoins et modélisation des cas d'utilisation (use case) 11
Conception de la structure objet avec le diagramme de classe 13
Élaboration du Modèle Conceptuel de Données (MCD) et du schéma physique de la base. 15
Conception de l'interface et des parcours utilisateur avec les wireframes et maquettes 17
Développement et implémentation technique 19
Méthode de travail 20
Mise en place de l'architecture applicative et des principes de la Clean Architecture 21
Création et mapping des entités avec Doctrine ORM 23
Implémentation de la logique métier et des cas d'usage 26
1. CreateDocumentUseCase 27
2. DocumentType 28
3. FileUploaderService 29
Développement des contrôleurs et gestion de routage 31
Conception et intégration de l'interface utilisateur (IHM) avec twig et Tailwind 34
Stratégie de tests unitaires 37
Sécurisation de l'application et veille technologique (authentification, autorisation, protection CSRF, etc.) 41
1. Gestion des accès : authentification et autorisation 41
2. Protection contre les attaques courantes 42
3. Gestion des données sensibles 43

Conclusion 43
Annexes 44

Introduction

Depuis mon enfance j'ai toujours été caractérisée par une profonde curiosité intellectuelle. J'ai été attirée par les sciences dès mon plus jeune âge, et animée par le désir constant de comprendre le fonctionnement du monde qui m'entourait.

Cette inclination m'a conduite à suivre un parcours très scientifique jusqu'au lycée. Mais une fois arrivée à l'université, c'est vers le droit que je me suis dirigée. Mon intérêt s'était porté vers la manière dont la société est régie et organisée. Cet intérêt et la passion pour les règles, et le fonctionnement de la société m'a permis d'obtenir une licence en droit.

À l'issue de cette licence, j'ai choisi d'effectuer une année de césure afin de prendre du recul et d'explorer d'autres domaines qui m'intéressaient, mais que je n'ai pas pu explorer à cause de mes études intensives.

Pendant cette césure, au-delà de la pâtisserie, c'est bien le développement web qui a monopolisé l'essentiel de mon temps.

Dès ma licence, l'informatique et son importance dans une société moderne comme la nôtre m'ont fortement intéressé. Avec l'avènement de l'intelligence artificielle et son impact sur nos vies, il était de plus en plus évident pour moi de devoir poursuivre un master en droit du numérique pour étudier ce changement de manière plus approfondie.

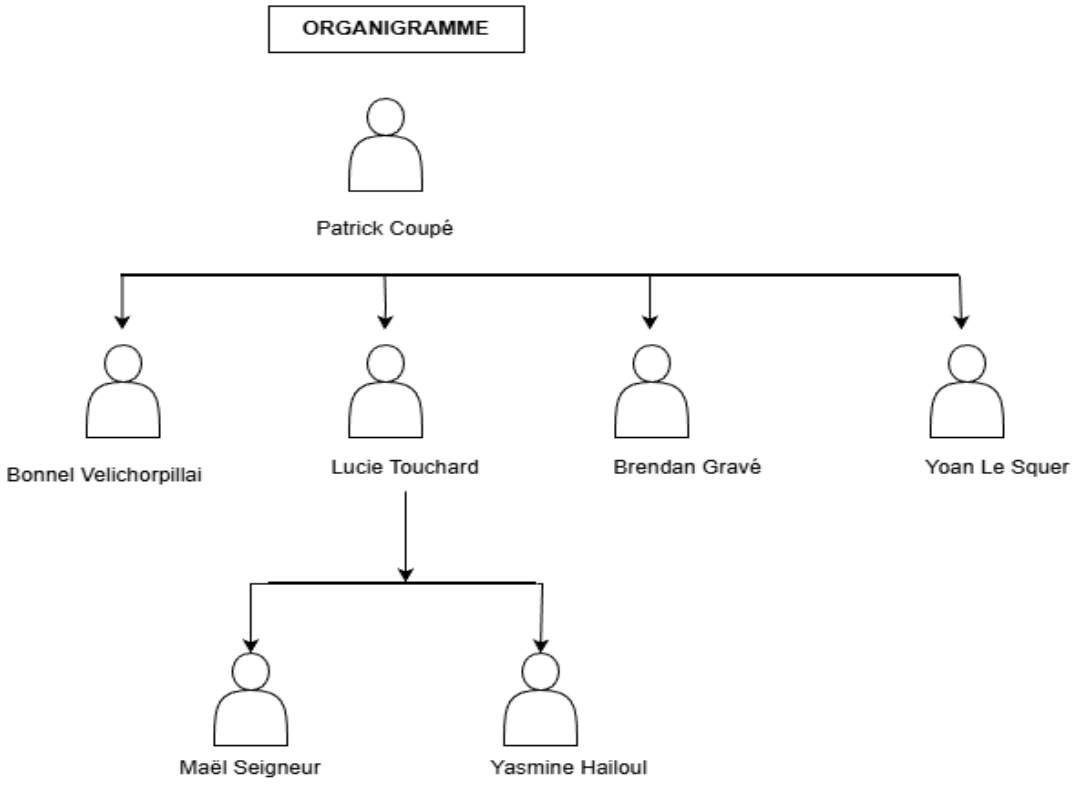
Mon année de césure, m'a permis de découvrir le monde du web à travers le développement, domaine fortement impacté par l'intelligence artificielle. J'ai fini par développer une passion pour ce processus, qui m'a poussé à suivre une formation en développement web et web mobile auprès de L'ENI pour acquérir les compétences techniques nécessaires à l'exercice de ce métier.

Mon projet professionnel s'inscrit dans la continuité de cette dynamique. Je souhaite en effet poursuivre mes études dans ce domaine et obtenir un master, dans l'objectif de devenir experte en architecture et développement de logiciels.

Découverte de l'établissement ASKORIA et de l'équipe informatique

Dans le cadre de ma formation de développement web et web mobile, j'ai effectué un stage de deux mois au sein de l'établissement ASKORIA. Cette institution privée est spécialisée dans la formation des métiers des secteur social, animation et médico-social. Cet établissement a vu le jour en 2013, après la fusion d'établissements de formation en travail social bretons. Grâce à cela, ASKORIA bénéficie d'une expertise de plus de 50 ans. Sa mission est de former des personnes dans le secteur du social, de l'animation et du médico-social. Cet établissement compte environ 250 employés.

Je suis rattachée au service informatique, doté d'une équipe composée de cinq personnes.



Mon tuteur de stage, Patrick Coupé est le directeur du service informatique chez ASKORIA et il occupe le rôle de testeur des applications produites par les développeurs.

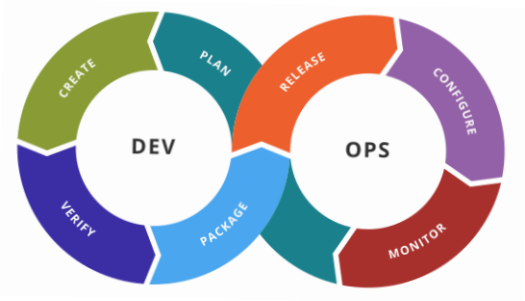
Brendan Gravé, Bonnel Velichorpillai, Yoan Le Squer sont des techniciens à casquettes multiples qui s'occupent du bon fonctionnement du réseau, du cloud et des servers des différents sites de l'établissement.

La responsable de développement web est Lucie Touchard. Elle s'occupe du développement de plusieurs applications internes et c'est sous sa responsabilité que j'ai réalisé mon stage avec un autre stagiaire Maël Seigneur.

Mise en place de l'environnement de développement

L'environnement de travail est équipé d'outils collaboratifs comme Git et GitLab.

Le premier jour a été dédié à la configuration de l'environnement de travail. Tous les outils étaient déjà installés mais la configuration était nécessaire pour utiliser l'environnement réseau. En effet pour le projet on suit l'approche DevOps, ce qui signifie que le développement et le déploiement sont intégrés dans un processus collaboratif et continu.



Pour ceci, on utilise GitLab. Il s'agit d'un outil de gestion de version de code et plateforme DevOps open source. La configuration de cet outil nécessite la génération d'une clé SSH. Le protocole SSH (Secure Shell) est utilisée lorsqu'on veut envoyer de manière sécurisée des commandes sur un réseau non sécurisé. Ce protocole implique la création de deux clés, une cryptographiée et l'autre non cryptographiée pour créer une connexion entre appareils. Cette connexion, ou tunnelisation, permet aux paquets de traverser des réseaux de manière sécurisée. Le protocole est donc utilisé pour contrôler des serveurs à distance, transférer des fichiers et gérer des infrastructures.

Outils, langages, systèmes

L'application codée lors du stage est codée entièrement en PHP (version 8.5), utilisant le Framework Symfony (version 7.4). La base de données est en Maria DB que nous avons accédé par le biais de Heidi SQL.

Après la configuration, ce qui a suivi, était une explication de la structure que le projet est obligé d'avoir. En effet, le code de l'application réalisée pendant ce stage suit le principe de la Clean Architecture. Il s'agit d'un patron de conception, basée sur les principes de programmation orientée objet qui a pour objectif de rendre le code compréhensible, flexible et maintenable.

Ce rapport va détailler le travail réalisé au sein de cet établissement en deux étapes. La première étape consiste à concevoir la structure de l'application. Ainsi, nous avons rédigé un cahier des charges détaillé incluant l'identification des besoins, la définition des types d'utilisateurs et la liste des fonctionnalités attendues. À partir de ce document, nous avons pu réaliser l'ensemble des schémas de conception nécessaires : le diagramme des cas d'utilisation (use case), diagramme d'activités pour les utilisateurs principaux, le diagramme de classe, le modèle conceptuel des données (MCD) ainsi que le schéma de base de données.

Dans un second temps, nous sommes passés à la phase de développement. En suivant les principes de la Clean Architecture, nous avons structuré le code de manière à séparer les couches métier, les services, contrôleurs et interface utilisateur.

Ce rapport présente l'ensemble de ces démarches, de l'analyse préliminaire jusqu'à l'obtention d'une interface homme-machine fonctionnelle.

Analyse et conception de l'application

L'application développée s'inscrit dans le cadre d'obligations contractuelles entre les intervenants et ASKORIA. Ainsi, les intervenants doivent fournir leurs supports de cours qu'ils utilisent pendant leurs interventions au sein d'ASKORIA et renseigner un compte rendu pour chaque intervention dispensée. Ces exigences sont la réponse aux obligations permettant l'obtention des certifications Qualiopi, essentielles pour assurer la perception de subventions qui servent à financer l'établissement.

Une fois les exigences du projets expliqués, nous avons commencé à réaliser les diagrammes de conception de l'application.

Cette étape est importante parce qu'elle permet de déterminer les fonctionnalités principales de l'application et leur enchainement. Pour cela, nous avons utilisé le logiciel Draw.io, conçu pour permettre la modélisation de diagrammes et schémas informatiques, ainsi que de maquettes.

Nous avons commencé par la création de l'organigramme de l'équipe informatique et le premier diagramme de l'application, le diagramme d'activité.

Nous avons pris en main TortoiseGit, outil permettant d'utiliser Git, sans devoir taper les lignes de commandes dans un terminal. La documentation a été essentielle à la compréhension du fonctionnement de cet outil et rendre le versioning du code grâce à Git plus simple.

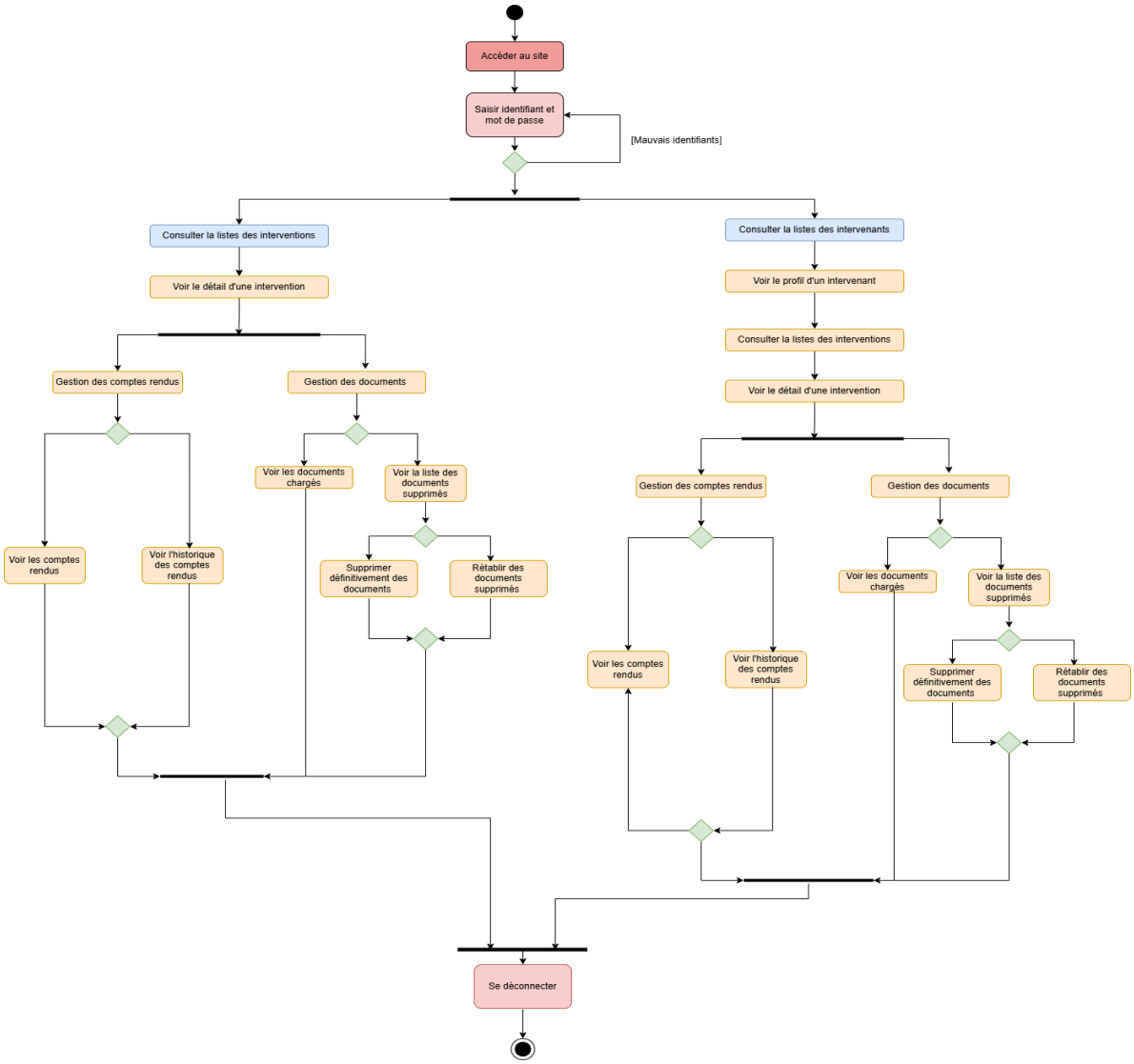
Nous avons pu nous entraîner à l'utilisation de TortoiseGit sur un autre projet test, mais aussi au codage d'une fonctionnalité dans une branche dédiée.

Modélisation des processus métier avec les diagrammes d'activité

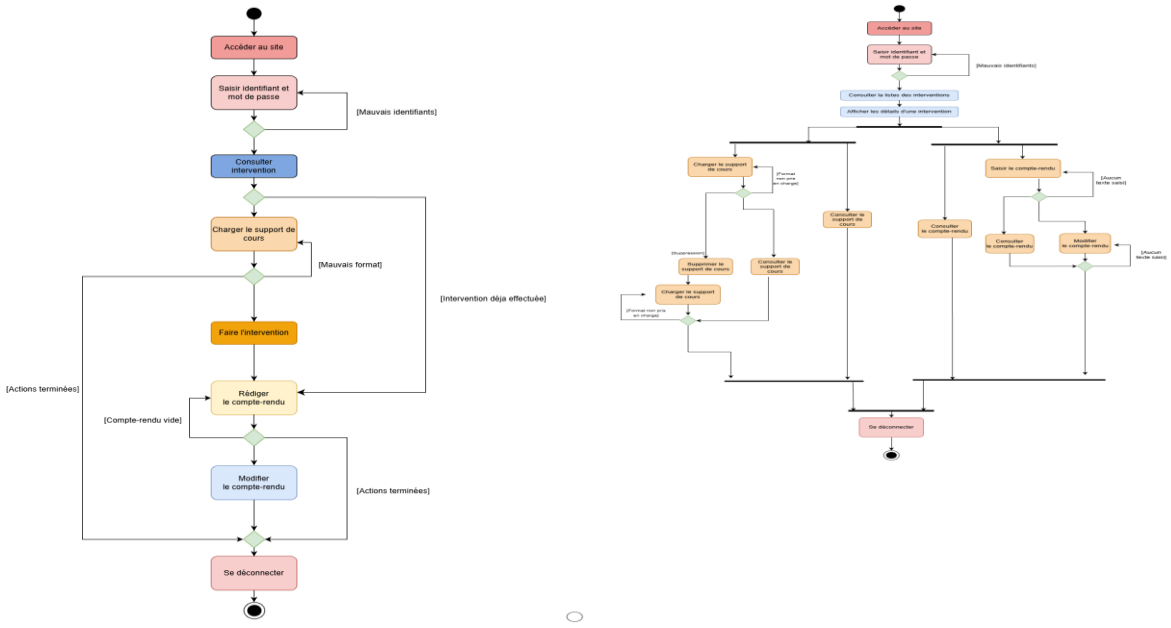
Comme mentionné précédemment, le premier diagramme de conception réalisé est le diagramme d'activité. Le but d'un diagramme d'activité est de décrire la séquence d'actions d'un processus. Il détaille les flux d'actions entre les différentes étapes.

Un diagramme d'activité ne se limite pas à décrire un enchaînement linéaire d'actions, mais aussi des parallélismes, (c'est à dire plusieurs actions simultanées), ou des branchements.

Dans notre cas, le diagramme d'activité décrit pour l'administrateur, l'enchainement d'actions qu'il peut réaliser, en utilisant l'application.



Ce diagramme a été réalisé sur plusieurs jours puisqu'il fallait soumettre les diagrammes à l'approbation de notre tutrice. En effet, lors de la réalisation des premiers diagrammes d'activités, nous avons essayé de simplifier les choses le plus possible, ignorant les parallélismes ou les branchements. Ce qui rendait les diagrammes incomplets et donc incorrects.



Dans l'ordre : la première version et la seconde version

En tout, pour la conception de l'application, nous avons proposé deux diagrammes d'activités. Le premier concerne le rôle d'administrateur, l'autre concerne le rôle d'intervenant, qu'il soit formateur interne ou externe à l'établissement ASKORIA.

L'enchaînement d'actions de l'intervenant est légèrement différent de celui de l'administrateur. En effet, bien que l'administrateur et l'intervenant soient tous deux des utilisateurs de l''application, ils occupent des rôles différents et réalisent donc des actions différentes.

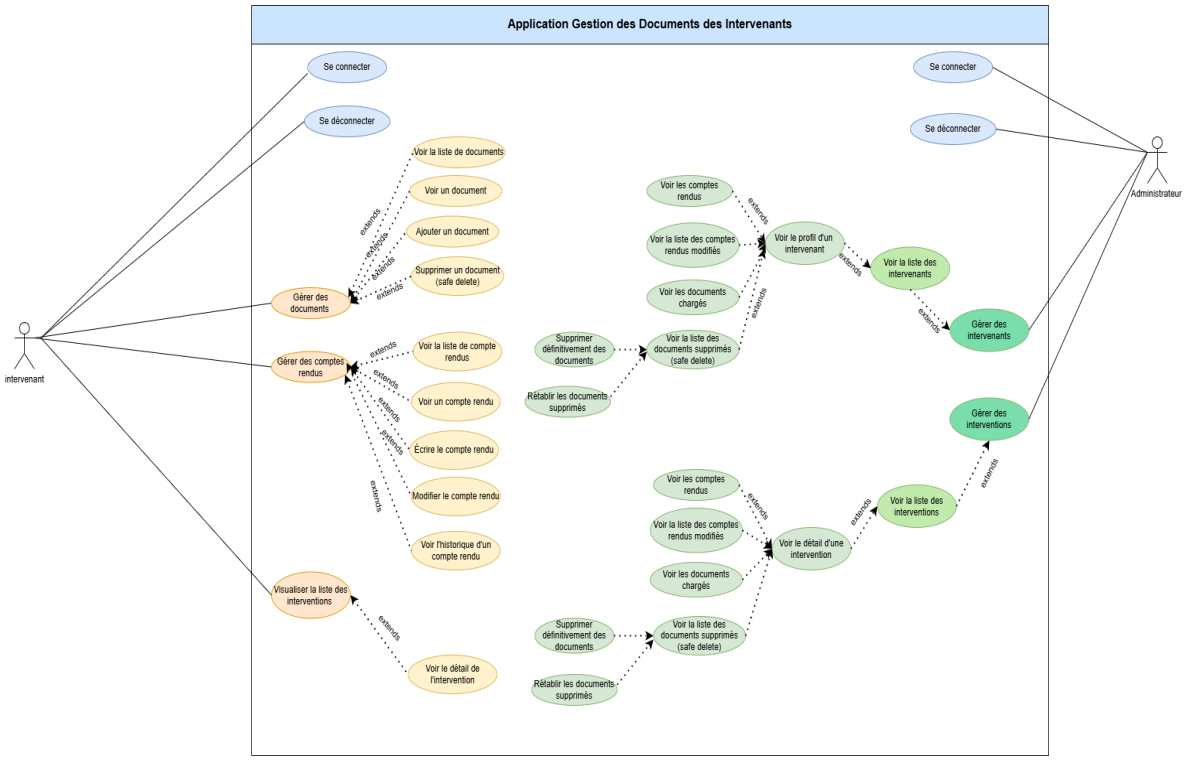
C'est pour cette raison que l'étape suivant dans la conception de l'application a été de réaliser le diagramme de cas d'utilisation.

Spécification des besoins et modélisation des cas d'utilisation (use case)

Les deuxième et troisièmes jours ont été dédiés à la réalisation de diagrammes. Le deuxième type de diagramme réalisé a été le diagramme de cas d'utilisation, plus communément appelé diagramme de use case.

En langage de modélisation unifié (UML), un diagramme de use case sert à définir de manière synthétique les utilisateurs d'un système et les interactions qu'ils ont avec ce dernier. Chaque action principale est définie, mais dans les diagrammes de use case les plus détaillés, les interactions principales avec le système peuvent être spécifiées. Dans ce cas, la relation entre l'action principale et la secondaire doit être précisée. Lorsque l'action secondaire est additionnelle et optionnelle on parle de "extends". Les conventions UML exigent alors que l'action secondaire soit liée à l'action principale par une flèche qui pointe vers l'action principale avec la mention "extends" écrite dessus.

Lorsque l'action secondaire a un caractère obligatoire, on parle de "includes". Il s'agit d'une action qui s'exécute à chaque fois que l'action de base est réalisée. Ceci est dû au fait que cette action secondaire est considérée comme essentielle à celle de base et sans laquelle l'action de base ne peut pas être réalisée.



Dans le cadre de l'application, le diagramme représente les acteurs principaux : l'intervenant et l'administrateur. Pour chacun, des actions de base sont définies et immédiatement la différence d'interaction avec le système est remarquée en fonction de l'acteur. L'administrateur joue un rôle de supervision, donc il est plus limité par rapport à l'intervenant, qui est le destinataire principal de cette application.

Comme pour le diagramme d'activité, il nous a fallu plusieurs jours pour pouvoir le modéliser correctement. Notre problème est resté le même pendant la phase de conception. Nous avons trop simplifié les diagrammes, ce qui a mené à des diagrammes imprécis.

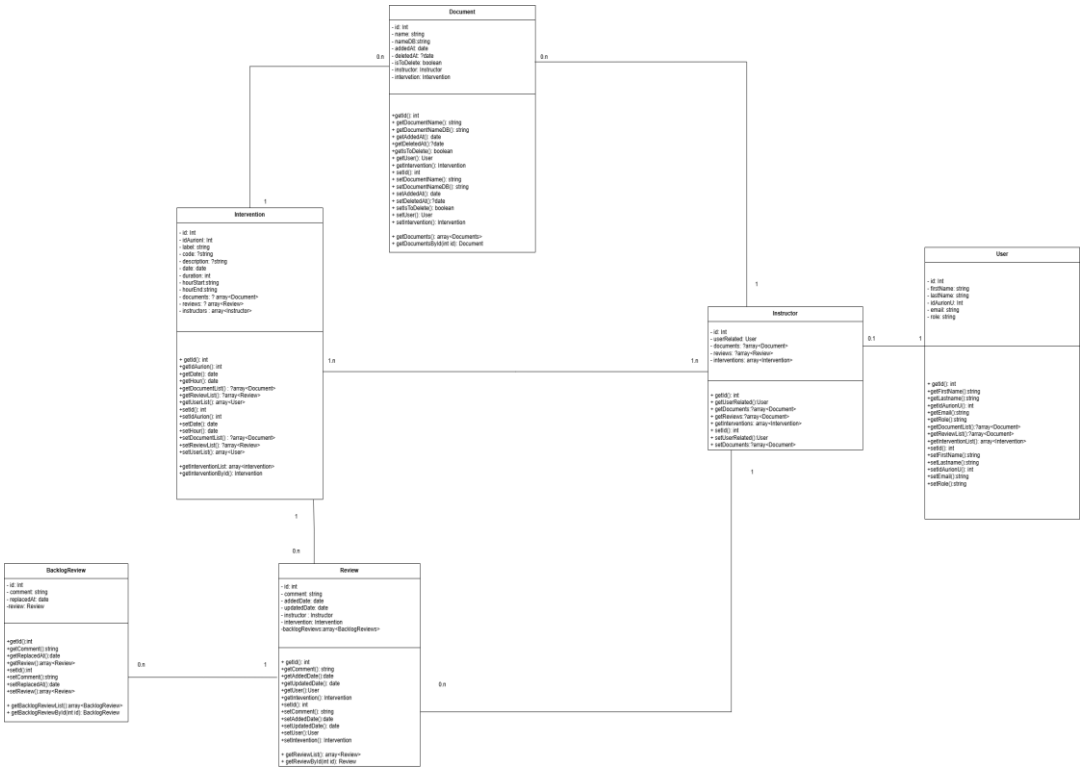
La lecture de la documentation, ainsi que les conseils de notre tutrice nous ont été essentiels à la correction du diagramme d'activité et des autres diagrammes.

La seconde partie du troisième jour a été consacrée à la réalisation des diagrammes de classe.

Conception de la structure objet avec le diagramme de classe

Une fois les fonctionnalités et les acteurs déterminés, nous avons pu passer à la modélisation du diagramme de classe.

Ce type de diagramme permet de visualiser des classes, des interfaces des systèmes ainsi que les relations entre celles-ci. Une classe décrit le comportement d'un ensemble d'objets "physiquement" par ses attributs et son comportement par la représentation de ses fonctions.



Dans notre cas, les classes principales sont User, Instructor, Intervention, Document, Review, et BacklogReview. Chacune de ces classes représente les composants essentiels du système de l'application. Comme on peut l'observer entre chaque classe il y a des relations d'association.

En UML, une relation d'association définit un lien conceptuel entre deux classes et les règles qui le régissent. Une des règles qu'on peut observer dans notre diagramme est la cardinalité qui permet de définir le type de relation.

En effet, la cardinalité permet de représenter le nombre minimum et maximum d'instances qui sont autorisées à participer à la relation. C'est grâce à la cardinalité qu'on peut parler de relation One to One, où une instance A est liée à une seule instance B et inversement. Ceci est le cas de la relation entre les classes User et instructor. Un objet Instructor ne peut qu'avoir une instance de l'objet User et vice versa.

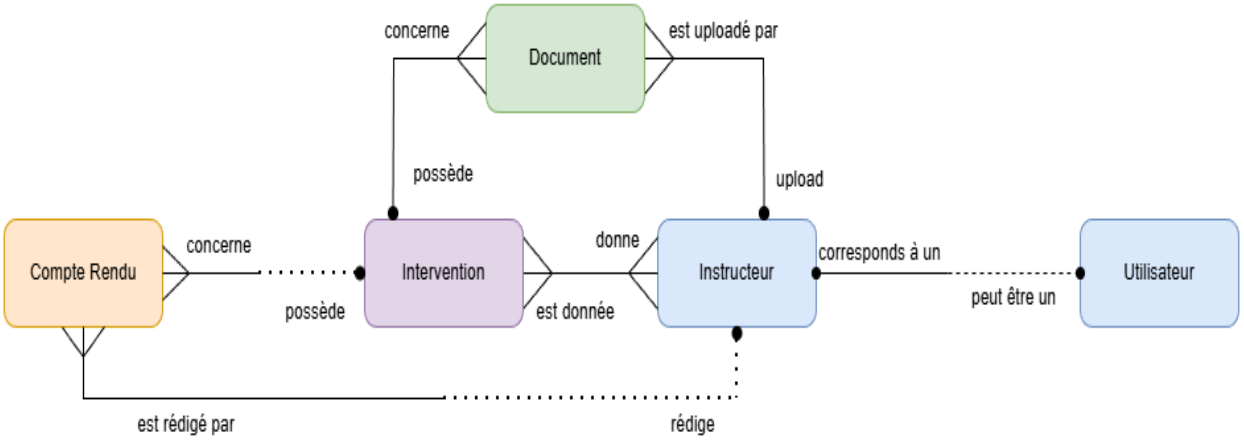
Un autre type de relation qu'on peut observer dans notre diagramme de classe est la relation One to Many, où une instance A est liée à plusieurs instances B, mais B n'est liée qu'à une seule instance A. Ce type de relation existe entre les classes Intervention et Document, Intervention et Review, entre Review et BacklogReview, e enfin entre Instructor et Document.

Le dernier type de relation défini dans le diagramme de classe est la relation Many to Many, où une instance de A a plusieurs instances de B et inversement. Ce type de relation existe entre les classes instructor et Intervention.

Élaboration du Modèle Conceptuel de Données (MCD) et du schéma physique de la base.

Une fois déterminées les entités et les relations entre elles, on a décidé de réaliser un Modèle Conceptuel de données (MCD). Le but était de formaliser la structure et Les informations recueillies. Le MCD représente ces informations de manière schématique, faisant abstraction des solutions et contraintes qu'on pourrait avoir lors de la structuration de la base de données. La raison pour laquelle on a réalisé un MCD c'est de savoir quelles informations stocker. Ça permet de faire valider les besoins métier et les règles de gestion.

Voici le MCD.



Le projet était de taille réduite (moins de 5-6 tables), avec des relations évidentes et sans contrainte métier complexe. Le passage direct au schéma physique n'a pas présenté de risque d'erreur. C'est pour cette raison qu'on n'a pas réalisé un MLD (modèle logique de données) et on a réalisé directement le schéma de base de données.

Un schéma de base de données est la représentation de l'organisation des données et des relations entre celles-ci au sein d'une base de données relationnelle. Le schéma démontre les contraintes logiques, tels que le nom de chaque table, les types de données et les relations entre chaque table.

La modélisation d'un schéma de base de données permet d'assurer la cohérence des données à travers toutes les contraintes mises en place.

Pour être plus précis, une table est un ensemble de données organisée.

Chaque table est structurée en colonnes, que l'on nomme aussi attributs. Ces colonnes contiennent des informations d'un type spécifique qui caractérisent chaque donnée enregistrée dans une ligne.

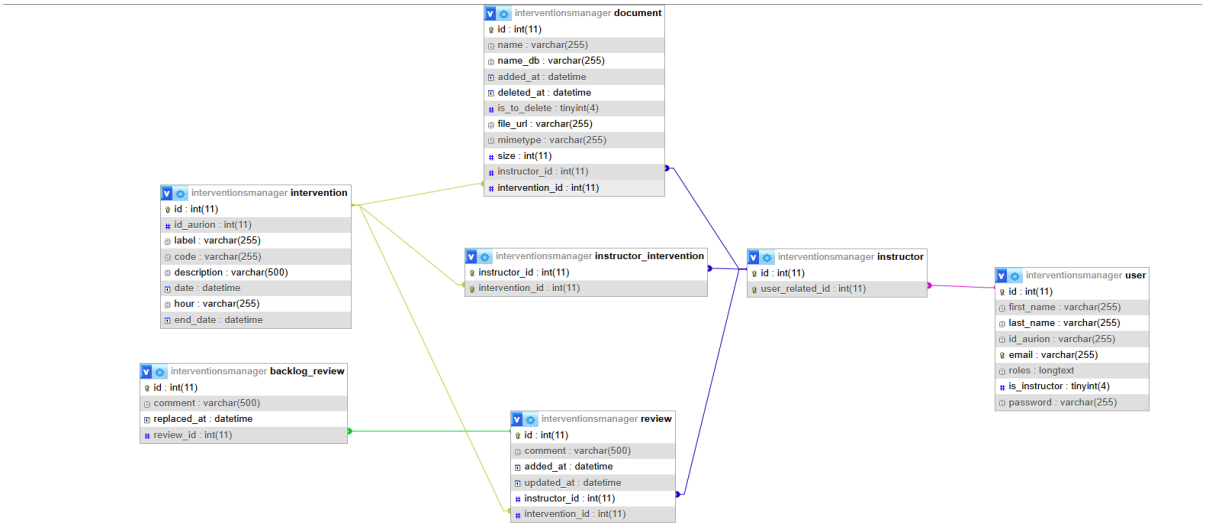
Chaque table est dotée d'une clé primaire (ou primary key), qui définit de façon unique chaque donnée enregistrée.

Une autre colonne qui va définir la donnée enregistrée dans la table est la clé étrangère. Il s'agit d'une référence à la clé primaire d'une autre table, et c'est par cet attribut qu'on peut définir les relations entre les tables.

Il y a des conventions concernant cette colonne. Quand il s'agit d'une relation One to Many, la colonne se trouve dans le côté Many. Dans le cas d'une relation Many to Many, les clés primaires des deux tables dans la relation se trouvent dans une table à part qu'on appelle table de pivot. Enfin, pour une relation One to One, la clé étrangère peut se trouver dans n'importe laquelle des deux directions de la relation.

On trouve d'autres règles régissant la construction d'une base de données, parmi celles-ci on trouve la contrainte d'unicité. Il s'agit d'une règle qui limite les valeurs possibles afin d'assurer l'intégrité des données. Les contraintes d'unicité sur une colonne interdisent des doublons dans la table.

Dans notre schéma de base de données, on observe l'application de ces règles et contraintes.



Le point focal de ce schéma sont les tables Intervention et Instructor, qui représente les formateurs. Chaque intervention peut avoir plusieurs formateurs et vice versa, comme on peut le voir au niveau de la table de pivot Intervention_Instructior.

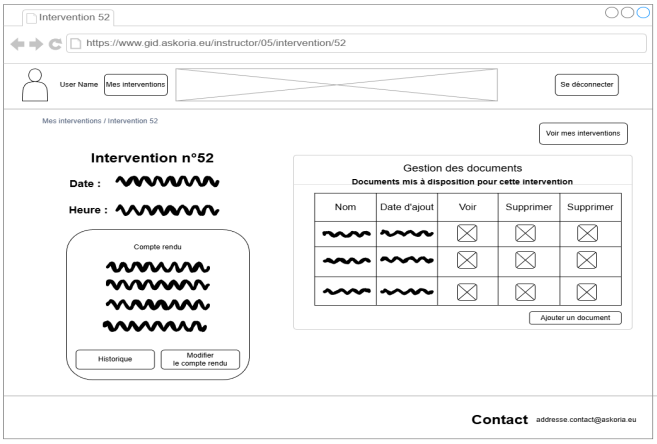
Chaque formateur correspond à un utilisateur, d'où la relation one to one entre les tables Instructor et User. La raison pour laquelle on a deux tables séparées c'est parce qu'un utilisateur peut ne pas être un formateur, mais un administrateur.

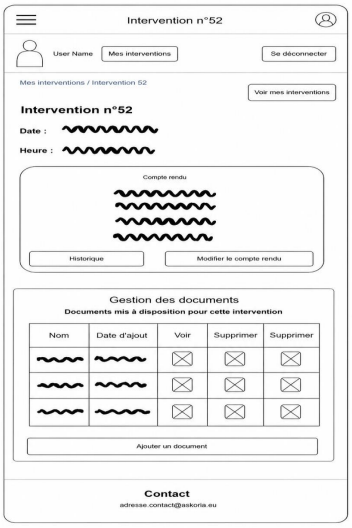
La raison pour laquelle cette distinction est nécessaire a déjà été précisée dans le diagramme de use case. Un administrateur n'a pas le même rôle qu'un formateur et donc ne peut pas réaliser les mêmes actions principales. On remarque aussi que dans la table User, il y a une contrainte d'unicité au niveau de la colonne email, qui sert à assurer qu'un utilisateur spécifique puisse accéder exclusivement à son compte.

Conception de l'interface et des parcours utilisateur avec les wireframes et maquettes

Une fois les diagrammes d'activité, le diagramme de cas d'utilisation, le diagramme de classe et le schéma de base de données réalisés, nous avons pu passer à l'étape suivante de la conception : la réalisation des wireframes.

Un wireframe est une représentation visuelle simplifiée de l'interface d'une application. Il s'agit d'un schéma en deux dimensions, généralement en noir et blanc ou en niveaux de gris, qui montre la structure de la page, l'agencement des éléments comme les boutons, les images, les blocs de texte, les formulaires et les menus de navigation. Les wireframes ne contiennent pas de couleurs, de polices stylisées, d'images finales ou de contenu rédactionnel définitif. Ils se concentrent uniquement sur la fonctionnalité et l'ergonomie, un peu comme un plan d'architecte pour un bâtiment. La réalisation des wireframes intervient pendant la phase de conception, après la collecte des besoins et avant la création des maquettes graphiques définitives et le développement. C'est une étape incontournable pour garantir que le produit final corresponde vraiment aux attentes des utilisateurs et aux objectifs du projet. Les wireframes servent de support de communication entre les différentes parties prenantes du projet, c'est-à-dire les chefs de projet, les designers, les développeurs et les clients. Un wireframe est bien plus facile à comprendre qu'un document technique, et il permet à tous de visualiser concrètement ce qui va être construit. Enfin, ils permettent de tester et de valider l'ergonomie et le parcours utilisateur à moindre coût. Il est beaucoup plus rapide et moins onéreux de modifier un wireframe que de changer du code ou une maquette graphique aboutie.





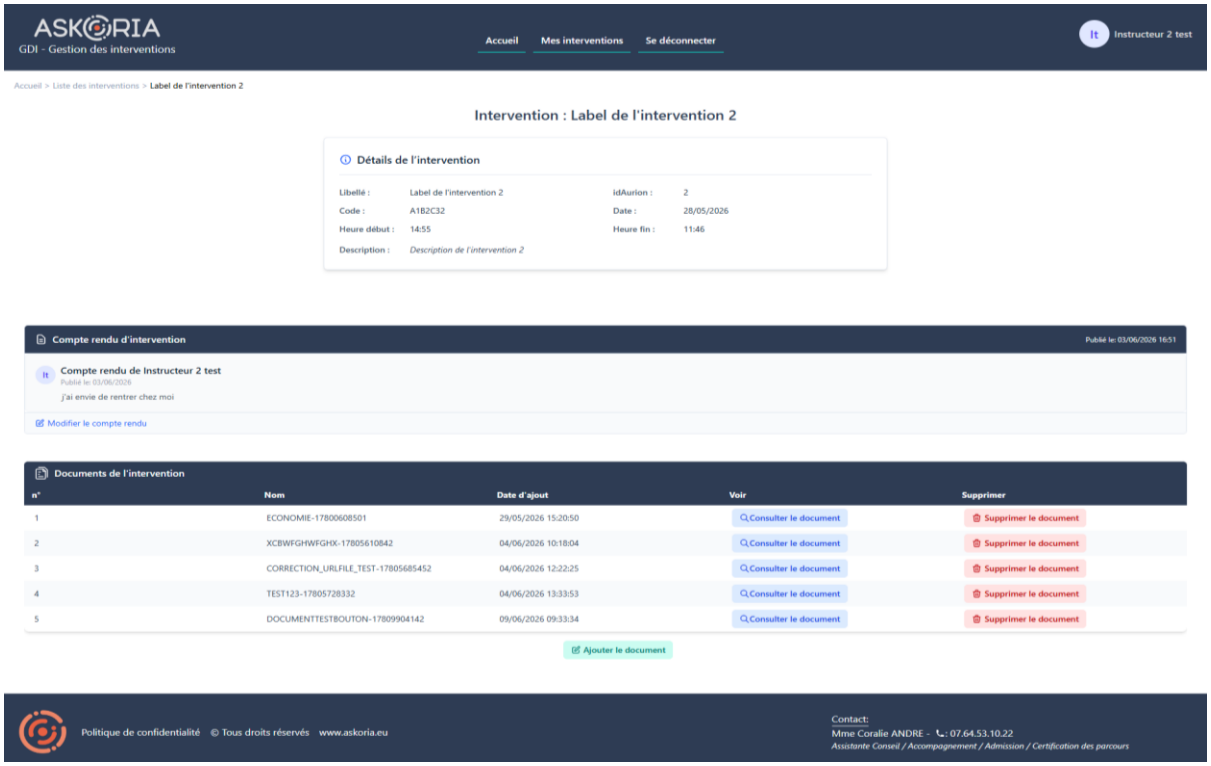
Dans le cadre de notre projet, la réalisation des wireframes a été une étape relativement rapide, puisqu'elle a été réalisée en une journée. Nous avons utilisé l'outil Draw.io, qui nous avait déjà servi pour la création des diagrammes de conception. L'objectif était de créer des maquettes basse fidélité pour chaque page principale de l'application, comme la page de connexion, le tableau de bord de l'intervenant, la page de gestion des interventions, la page de dépôt de documents et la page de dépôt des comptes rendus. L'application est uniquement utilisée sur PC, mais nous avons trouvé pertinent de montrer à quoi ressemblerait l'application

en version mobile. La page représentée dans les deux wireframes correspond à la page du détail d'une intervention appartenant à un formateur.

À la fin de cette étape, nous avons présenté les wireframes à notre tutrice pour validation. Cette présentation a permis de confirmer que la structure de l'application était conforme aux exigences fonctionnelles identifiées lors des étapes précédentes. À partir des wireframes nous avons pu développeur les maquettes de l'application.

Une maquette est une représentation visuelle plus aboutie que le wireframe. Contrairement au wireframe qui est généralement en noir et blanc et se concentre sur la structure et l'ergonomie, la maquette intègre les couleurs, les typographies, les images et les icônes définitives du projet. Elle donne une vision beaucoup plus réaliste du rendu final de l'application ou du site web. La maquette est réalisée après la validation des wireframes, lors de la phase de conception graphique. Elle sert à valider l'identité visuelle et l'aspect esthétique du projet auprès du client avant de passer au développement. C'est un outil de communication essentiel entre les designers et les développeurs, car elle permet à ces derniers de savoir précisément comment doit être implémentée l'interface utilisateur.

Voici la maquette de la page du détail d'une intervention appartenant à un formateur.



Cette étape m'a permis de comprendre l'importance de la conception visuelle avant le développement. En effet, en tant que développeuse, il est facile de se concentrer uniquement sur le code et la logique métier, mais un projet ne peut réussir que si l'interface est intuitive et ergonomique pour l'utilisateur final. Les wireframes sont donc un outil précieux qui fait le pont entre les spécifications techniques et l'expérience utilisateur.

Développement et implémentation technique

Méthode de travail

L'étape de développement est celle qui a occupé la majorité du temps lors de la réalisation du stage.

Des règles ont caractérisé cette phase.

Comme nous l'avons déjà mentionné avant, la méthode de développement suit l'approche DevOps, qui consiste en un développement et déploiement constant et continu dans le cadre d'un processus collaboratif. Cette exigence a été assurée par GitLab. Il s'agit d'un outil de gestion de versions de code qui met à disposition un environnement collaboratif.

Le processus de développement a été automatisé avec des pipelines. Un pipeline est l'ensemble d'outils et de processus permettant à toutes les personnes travaillant sur le projet, de collaborer sur l'environnement de production. Pour assurer que le code soit exploitable, des outils sont utilisés lors de l'exécution du pipeline.

Le premier outil qu'on a utilisé est PHP Code Sniffer, ou phpcs. Il s'occupe de détecter les violations définies par le standard de développement de PHP, permettant d'améliorer la lisibilité et la compréhensibilité du code.

Le deuxième outil est PHPStan, outil d'analyse statique qui permet de détecter des bugs. Si pendant l'exécution du pipeline, une infraction est détectée par l'un de ces deux outils, le pipeline s'arrête et ne pourra reprendre que si les infractions sont corrigées. Les pipelines permettent une intégration continue (CI) et un déploiement continu (CD).

L'intégration continue automatise les tests à chaque modification. Dans notre workflow, chaque fonctionnalité est développée dans une branche dédiée. Les tests que la branche subit à chaque modification sont les contrôles de phpstan et phpcs . Une fois les tests passés et la fonctionnalité est terminée, elle est intégrée dans un référentiel partagé (la branche main) dans Git.

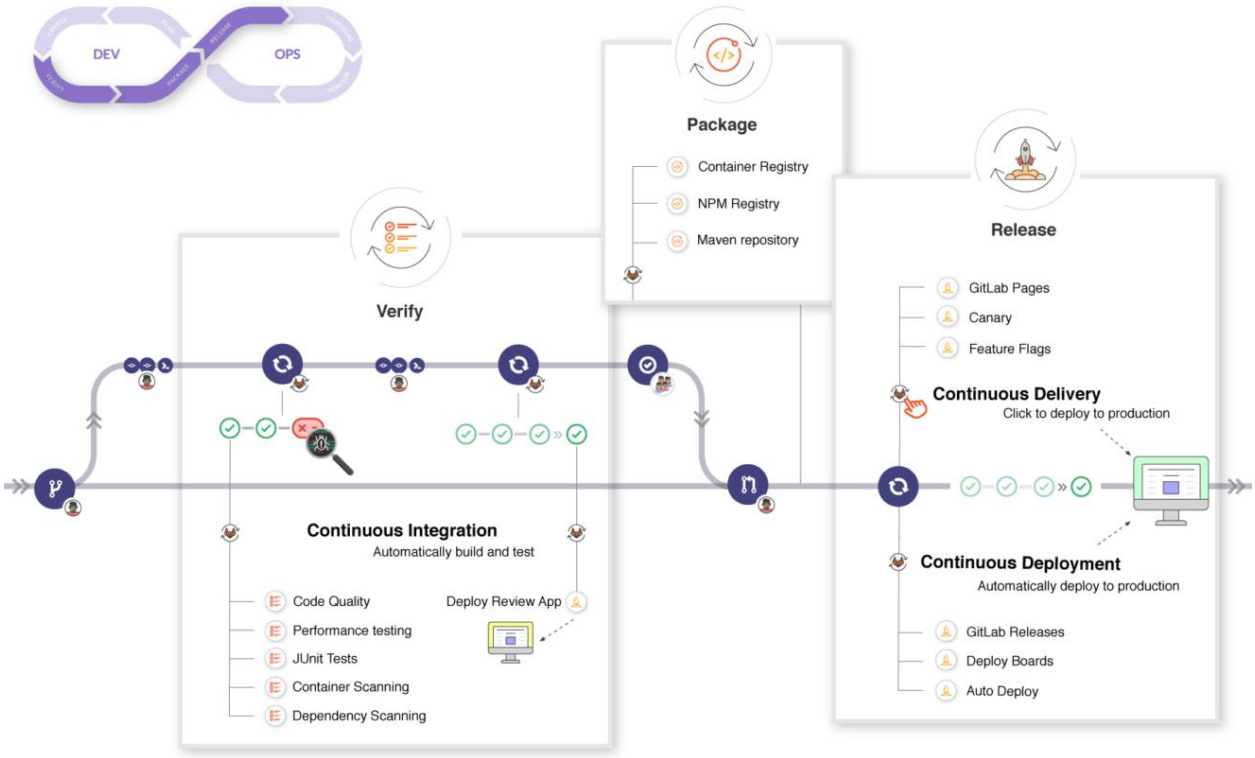
Notre workflow de développement comprend plusieurs étapes.

L'étape préliminaire à la phase de développement est la création de issues. Une issue permet de déclarer et suivre une fonctionnalité à réaliser lors du développement de l'application. Ce n'est qu'à la fin de la création d'une issue qu'on peut créer la branche, (via une merge request) pour développer la fonctionnalité.

Une fois que nous récupérons la branche en local, sur notre machine, nous pouvons coder jusqu'à ce que la fonctionnalité soit complétée et fonctionnelle. Ce n'est qu'à ce moment qu'on pourra procéder à l'étape d'intégration. Si tous les contrôles passent, la branche est fusionnée à la branche principale sur Git, permettant aux autres personnes travaillant sur le projet de récupérer la fonctionnalité.

La phase de développement continu pousse l'automatisation à son terme. En effet, toute modification apportée au code qui réussit toutes les étapes de contrôles, est automatiquement déployée en production, sans avoir besoin d'intervention humaine. Cela a pour avantage de rendre le processus de production plus fluide et de permettre des mises à jour du code plus facilement.

Grâce aux test réalisées en amont, l'identification de problèmes et plus rapide, ce qui permet de diminuer les risques car la partie du code présentant un problème sera facilement ciblée et donc facilement corrigée.



Mise en place de l'architecture applicative et des principes de la Clean Architecture

L'architecture de l'application se base sur les principes de la Clean Architecture.

La Clean Architecture est une philosophie de conception de logiciels basée sur la logique métier de l'application. L'objectif est d'assurer la séparation claire des responsabilités du code, de limiter les dépendances externes et de garantir une forte testabilité. Une telle structure limite considérablement la dette technique, c'est-à-dire les coûts futurs associés à des compromis qu'on aurait pris lors du développement initial, qui génèrent des coûts additionnels dans le cas de la survenue de bugs ou dans le cas de l'évolution de l'application. Par nature, la Clean Architecture permet de faciliter les évolutions fonctionnelles de l'application.

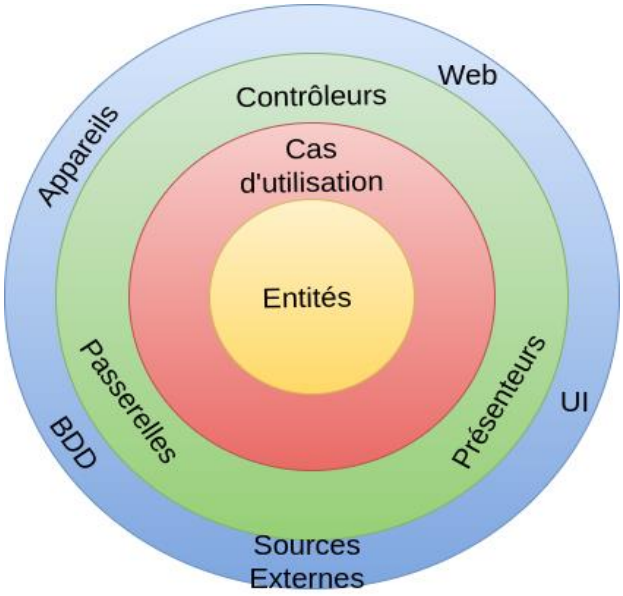
Grâce à ses avantages, beaucoup de grandes entreprises utilisent dans leur applications ce type d'architecture.

Cette architecture comporte trois principes.

Le premier principe, et le plus fondamental, est la séparation des responsabilités.

Chaque couche de l'architecture possède un rôle spécifique et distinct de celui des autres couches. Un changement d'une couche ne doit avoir en théorie aucun effet sur les autres couches.

Il y a quatre couches indépendantes :



Les entités (ou entities) qui correspondent à l'objet du domaine métier. Elles sont indépendantes et peuvent être utilisées par toutes les couches de l'application.

Les cas d'usage ou use case qui enferment toute la logique métier de l'application. Chaque use case représente une règle métier spécifique que l'un des acteurs de l'application sera amené à réaliser. Les use cases interagissent avec les entités et déterminent la manière dont les données sont transférées aux couches extérieures.

La troisième couche contient les adaptateurs d'interface et les contrôleurs, aussi connus sous le nom de présenteurs ou passerelles, parce qu'ils mettent en contact les use case avec les couches plus externes. Ils ont le rôle de traduire les données transférées entre les couches externes ou l'utilisateur et les use cases.

La dernière couche inclut tout ce qui est en contact avec le "monde extérieur", tel que les interfaces utilisateur, le système de base de données, les serveurs web, API etc... Le principe d'indépendance assure que la logique de l'application et de façon plus large, l'application elle-même ne soit pas dépendante de cette dernière couche, donc de toute implémentation technique. Cette indépendance rend facile de remplacer des composants de cette dernière couche, ce qui permet la longévité de l'application malgré les évolutions techniques.

Le dernier principe est le couplage faible et la forte testabilité. En effet, la séparation plus ou moins stricte des préoccupations permet de tester les composants de manière isolée et donc plus facile.

Création et mapping des entités avec Doctrine ORM

L'architecture de l'application se construit en commençant par la mise en place des entités.

Une entité est une classe PHP qui peut être mappée avec une table dans la base de données grâce à un ORM. Un ORM (Object-Relation Mapping) est une couche d'abstraction entre une base de données relationnelle et dans ce cas un programme PHP orienté objet. Un ORM sert à créer une correspondance entre un modèle objet et un modèle relationnel de base de données.

Puisque nous avons travaillé avec l'ORM de Symfony, Doctrine, avant de créer les entités on a dû configurer la base de données au niveau du fichier .env de notre projet

```
# DATABASE_URL="sqlite:///%kernel.project_dir%/var/data_%kernel.environment%.db"
DATABASE_URL="mysql://root:@127.0.0.1:3306/interventionsmanager"
# DATABASE_URL="mysql://app:!ChangeMe!@127.0.0.1:3306/app?serverVersion=10.11.2-MariaDB&charset=utf8mb4"
#DATABASE_URL="postgresql://app:!ChangeMe!@127.0.0.1:5432/app?serverVersion=16&charset=utf8"
###< doctrine/doctrine-bundle ###
```

Pour des raisons de sécurité la configuration a été modifiée pour omettre le mot de passe. Une fois la base de données configurée, on a pu créer les entités avec cette commande

```
yasmine.hailoul@RNS-P-L5520-YHA MINGW64 ~/Documents/Repositories/Applications/interventionsmanager (43-new-creation-vue-backlogreviews-2)
$ make entity
/usr/bin/sh: line 1: docker: command not found
php bin/console make:entity

 Class name of the entity to create or update (e.g. BravePizza):
 > Document
```

L'ORM nous guide à la création de chaque attribut de l'entité en requérant le type, la taille maximale d'un attribut de type string et la nullabilité. (cf Annexe n* x pour voir la création complète d'une entité). Lorsqu'on crée toutes nos entités on pourra, par l'intermédiaire de l'ORM,ajouter les attributs représentant les relations entre les tables. Le processus est toujours le même, il faut utiliser la commande `php bin/console make:entity` et entrer le nom de la table dans laquelle on veut ajouter un attribut et à la place de définir un type, on définit une relation avec une table et le type de relation (cf. annexe n° x pour voir création d'une relation avec une table). Une fois que la création des entités est complète on exécute des commandes pour créer les tables correspondantes dans la base de données `php bin/console make:migration` et `php bin/console doctrine:migrations:migrate`

```
yasmine.hailoul@RNS-P-L5520-YHA MINGW64 ~/Documents/Repositories/Applications/interventionsmanager (4-new-creation-des-entites-apres-betise)
$ make migration
/usr/bin/sh: line 1: docker: command not found
php bin/console make:migration
 created: migrations/Version20260430124503.php

 Success!

 Review the new migration then run it with php bin/console doctrine:migrations:migrate
 See https://symfony.com/doc/current/bundles/DoctrineMigrationsBundle/index.html

yasmine.hailoul@RNS-P-L5520-YHA MINGW64 ~/Documents/Repositories/Applications/interventionsmanager (4-new-creation-des-entites-apres-betise)
$ make migration-migrate
/usr/bin/sh: line 1: docker: command not found
## Execute migrations...
php bin/console doctrine:migrations:migrate

 WARNING! You are about to execute a migration in database "interventionsmanager" that could result in schema changes and data loss. Are you sure you wish to continue? (yes/no) [yes]:
 >

[notice] Migrating up to DoctrineMigrations\Version20260430124503
[notice] finished in 1424.1ms, used 22M memory, 1 migrations executed, 16 sql queries

[OK] Successfully migrated to version: DoctrineMigrations\Version20260430124503
```

Automatiquement, ma base de données est construite avec les tables nécessaires respectant les contraintes représentées dans le schéma de base de données.

Si Symfony n'utilisait pas l'ORM Doctrine, nous aurions créé la base utilisant des requêtes SQL comme celle-ci :

```sql
CREATE TABLE document (
    id INT AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    name_db VARCHAR(255) NOT NULL,
    added_at DATETIME NOT NULL,
    deleted_at DATETIME DEFAULT NULL,
    is_to_delete TINYINT DEFAULT NULL,
    instructor_id INT DEFAULT NULL,
    intervention_id INT DEFAULT NULL,
    PRIMARY KEY (id)
);
ALTER TABLE document ADD CONSTRAINT instructor_id FOREIGN KEY (instructor_id) REFERENCES instructor (id);
ALTER TABLE document ADD CONSTRAINT intervention_id FOREIGN KEY (intervention_id) REFERENCES intervention (id);
```

Dans le cadre de notre application, les entités ne sont qu'une partie composant le domaine.

Un domaine est la couche qui englobe les entités et la couche métier. Dans le domaine nous pouvons trouver les DTO (Data Transfer Object ou les objets de transfert de données), des objets qui servent à transférer des données à travers les processus entamés par les use cases du domaine. Un DTO assure le transfert uniquement des données qui nous intéressent, ce qui ajoute une couche additionnelle de sécurité, car l'utilisation directe d'une entité aurait impliqué le transfert de toutes les informations liées à l'entité, figurant dans la base de données.

```php
class DocumentModelData
{
    public ?int $id = null;
    public ?string $name = null;
    1 usage
    public ?string $nameDB = null;
    public ?\DateTimeImmutable $addedAt = null;
    1 usage
    public ?\DateTimeImmutable $deletedAt = null;
    public ?bool $isToDelete = null;
    public ?InstructorModelData $instructor = null;
    public ?int $instructorId = null;
    public ?InterventionModelData $intervention = null;
    public ?int $interventionId = null;
    3 usages
    public ?string $fileURL = null;
    public ?UploadedFile $file = null;
    public ?int $size = null;
    public ?string $mimetype = null;

    & Hailoul
    public static function createFromEntity(Document $document): self
    {
        $self = new self();
        $self->id = $document->getId();
        $self->name = $document->getName();
        $self->nameDB = $document->getNameDB();
        $self->addedAt = $document->getAddedAt();
        $self->deletedAt = $document->getDeletedAt();
        $self->isToDelete = $document->isToDelete();
        $self->fileURL = $document->getFileURL();
        $self->size = $document->getSize();
        $self->mimetype = $document->getMimetype();

        return $self;
    }
}
```

C’est pour cette raison que dans les use case on ne travaille qu’avec des DTO.

Implémentation de la logique métier et des cas d'usage

Comme nous l'avons mentionné avant, l'utilisation des DTO est fondamentale, et dans presque tous les cas on utilise ce patron de conception pour créer la logique métier.

Les fonctionnalités de l'applications sont nombreuses, mais nous avons fait le choix de nous concentrer sur la fonctionnalité d'upload de documents. Cette fonctionnalité permet à un formateur d'uploader un document, comme le support de son cours, au niveau de l'interface d'une intervention particulière.

1. CreateDocumentUseCase

La fonctionnalité doit être capable de réaliser deux choses : insérer en base de données les informations concernant le document, et réaliser l'action d'upload, c'est à dire, télécharger un document depuis son ordinateur vers l'application.

Voici le code de la fonction :

```php
    /**
     * @throws CannotCreateDocumentException
     * @throws Throwable
     */
    public function create(
        DocumentModelData $document,
        int $instructorId,
        int $interventionId
    ): void {
        $name = strtoupper(trim($document->name));
        if (preg_match('/^[a-zA-Z0-9_-]+$/', $name) == 0) {
            throw new CannotCreateDocumentException("Seuls les caractères spéciaux suivants sont admis : - et _");
        }

        $intervention = $this->interventionRepository->find($interventionId);
        $instructor = $this->instructorRepository->find($instructorId);
        $interventionCode = $intervention->getCode();
        $file = $document->file;
        $fileURL = "public/uploads/" . $name . "-" . time() . $instructorId . '.' . $file->guessExtension();
        $newDocument = new Document();
        $newDocument->setName($name . "-" . time() . $instructorId);
        $newDocument->setNameDB($instructorId . $interventionId . $interventionCode);
        $newDocument->setAddedAt(new \DateTimeImmutable());
        $newDocument->setIsToDelete(false);
        $newDocument->setDeletedAt(null);
        $newDocument->setInstructor($instructor);
        $newDocument->setIntervention($intervention);
        $newDocument->setFileURL($fileURL);
        $newDocument->setSize($this->fileUploaderService->fileSize($file));
        $newDocument->setMimetype($this->fileUploaderService->mimetype($file));

        try {
            $this->fileUploaderService->upload($file, $newDocument->getName());
            $this->entityManager->persist($newDocument);
            $this->entityManager->flush();
        } catch (Throwable $e) {
            throw new CannotCreateDocumentException("le document ne peut pas être importé : " . $e->getMessage());
        }
    }
}

```

Étant donné qu'un document est lié à un formateur et à une intervention précise, on aura besoin de l'id du formateur et de l'intervention, mais aussi des données liées au document uploadé qui sont contenues dans un DTO.

La première partie de cette fonction s'assure que le nom du document saisi par le formateur est conforme à la règle mise en place.

Si la règle n'est pas respectée une exception se déclenche, ce qui empêche l'exécution de la fonction create. De plus, l'erreur remonte à l'utilisateur qui sera informé de cette inconformité.

La suite du code, entre la ligne 40 et 55 servent à définir les informations qui seront insérées dans la base de données. Les lignes de code qui suivent, sont les plus importantes. En effet, la ligne 58, fait appel à un service qui se charge d'uploader le document, et les deux lignes suivantes, insèrent les informations liées au document dans la base de données.

2. *DocumentType*

L'étape préliminaire à l'exécution de la fonction create de la classe CreateDocumentUseCase corresponds à la création d'un formulaire qui va envoyer le document au server. La création d'un formulaire commence par la ligne de commande `php bin/console make:form` .

```
yasmine.hailoul@RNS-P-L5520-YHA MINGW64 ~/Documents/Repositories/Applications/interventionsmanager (8-new-creati
on-create-document)
$ php bin/console make:form

 The name of the form class (e.g. DeliciousPuppyType):
 > Document

 The name of Entity or fully qualified model class name that the new form will be bound to (empty for none):
 > Document
Document

 created: src/Form/DocumentType.php

 Success!
```

Cette commande permet de générer automatiquement un formulaire basique associé à une entité. Dans notre cas, on a associé le formulaire à l'entité Document.

Le formulaire est modulable car nous pouvons ajouter dans le formulaire uniquement les champs que nous voulons rendre modifiables. En effet, il ne serait pas pertinent d'ajouter les champs concernant les dates, comme $addedAt qui, pour des soucis de confort de l'utilisateur, doivent être renseignés de manière automatique.

D'autres champs ne peuvent pas être remplis lors de l'upload du document, simplement parce qu'on pourra récupérer l'information après l'upload.

Voici le formulaire d'upload de documents :

```php
class DocumentType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('name', TextType::class, [
                'label' => 'Nom du document',
                'attr' => [
                    'placeholder' => 'Saisissez le nom du document',
                    'required' => true,
                ],
                'constraints' => [
                    new NotBlank(message: "Veuillez saisir le nom du document"),
                    new NotNull(message: "Veuillez saisir le nom du document"),
                    new Regex(
                        pattern: '/^[a-zA-Z0-9_-]+$/',
                        message: 'Le nom ne peut pas contenir d\'accents ni de caractères spéciaux autres que _ ou -'
                    ),
                ]
            ])
            ->add('file', FileType::class, [
                'label' => 'Choisissez un document',
                'attr' => [
                    'placeholder' => 'Veuillez choisir un document',
                    'required' => true,
                ],
                'constraints' => [
                    new File(
                        maxSize: '5M',
                        maxSizeMessage: 'Veuillez choisir un document dont la taille est inférieure à 5 Mo',
                        uploadErrorMessage: 'Une erreur s\'est produite lors de l\'import du document',
                        extensions: ['pdf', 'doc', 'docx', 'ppt', 'pptx', 'xls', 'xlsx'],
                        extensionsMessage: "Le type du document doit être : pdf, doc, docx, ppt, pptx, xls, xlsx",
                    )
                ]
            ])
            ->add('submit', SubmitType::class, [
                'label' => 'Importer le document',
            ])
            ->add('reset', ResetType::class, [
                'label' => 'Annuler',
            ]);
    }

    public function configureOptions(OptionsResolver $resolver): void
    {
        $resolver->setDefaults([
            'data_class' => DocumentModelData::class,
        ]);
    }
}
```

Les seuls champs que nous avons permis aux utilisateurs à remplir sont le nom qu'ils souhaitent donner au document, et le champ file, qui permet d'uploader le document. On peut constater que les deux champs sont régis par des règles.

Concernant le champ du nom, il est interdit que le nom contienne des caractères spéciaux autres que – ou _. Parmi les caractères interdits il y aussi les lettres accentuées pour éviter des problèmes d'affichage. Une autre règle imposée dans le formulaire concerne les fichiers eux-mêmes. On a une limite de taille et une limite de types de documents acceptés.

Ces règles sont en place pour limiter la charge sur la base de données, mais aussi pour s'assurer que le type de document uploadé peut être confortablement lu par les administrateurs du site, ainsi que par le personnel de Qualiopi, dans le cas où ils voudraient les récupérer. Si on permettait tout type de document, on n'aurait pas la capacité d'assurer la lecture du fichier par tous.

3. *FileUploaderService*

Sans la présence du FileUploaderService, l'upload de documents ne serait pas possible.

C'est un service qui utilise principalement des fonctions natives au Framework Symfony (cf. les fonctions guessExtension() et move()).

```php
class FileUploaderService
{
    public function __construct(
        private string $targetDirectory,
    ) {
    }

    public function upload(UploadedFile $file, string $name): string
    {
        $fileName = $name . '.' . $file->guessExtension();
        try {
            $file->move($this->targetDirectory, $fileName);
        } catch (FileException $e) {
            throw new FileException("une erreur est survenue lors de l'upload du fichier.");
        }
        return $fileName;
    }

```

Pour que le service puisse marcher on doit d'abord configurer la $targetDirectory, c'est-à-dire le nom du répertoire où les documents uploadés seront stockés.

services.yaml 1.35 KiB

```yaml
# yaml-language-server: $schema=../vendor/symfony/dependency-injection/Loader/schema/services.schema.json

# This file is the entry point to configure your own services.
# Files in the packages/ subdirectory configure your dependencies.
# See also https://symfony.com/doc/current/service_container/import.html

# Put parameters here that don't need to change on each machine where the app is deployed
# https://symfony.com/doc/current/best_practices.html#use-parameters-for-application-configuration
parameters:
    uploads_directory: '%kernel.project_dir%/public/uploads'

services:
    # default configuration for services in *this* file
    _defaults:
        autowire: true      # Automatically injects dependencies in your services.
        autoconfigure: true # Automatically registers your services as commands, event subscribers, etc.

    # makes classes in src/ available to be used as services
    # this creates a service per class whose id is the fully-qualified class name
    App\:
        resource: '../src/'

    # add more service definitions when explicit configuration is needed
    # please note that last definitions always *replace* previous ones
    App\Service\FileUploaderService:
        arguments:
            $targetDirectory: '%uploads_directory%'
    App\Service\FileRemoverService:
        arguments:
            $targetDirectory : '%uploads_directory%'
```

Dans le cadre de la création de documents, la difficulté principale était de comprendre quelle méthode était idéale pour permettre l'upload des documents.

On a choisi cette méthode parce qu'une fois l'application déployée, les documents uploadés seraient stockés dans le serveur.

Grâce à cette méthode, il n'y a pas une limite au nombre de documents à uploader par intervention, mais surtout on permet aux formateurs d'uploader plusieurs types de fichiers.

Au début, nous avons décidé de stocker directement les documents dans la base de données en définissant un colonne nommée file, de type LONGBLOB.

Or cette méthode présentait beaucoup d'inconvénients.

En effet, pour ne pas saturer la base de données, la taille du document uploadé devait être considérablement plus réduite. De plus, il n'y avait que très peu de types de fichiers tolérés par la base de données. Ainsi, uniquement les fichiers de type image et les fichiers de type PDF pouvaient être stockés.

Malgré le fait que cette fonctionnalité ait pris plusieurs tentatives, j'ai appris comment stocker directement des fichiers dans la base de données.

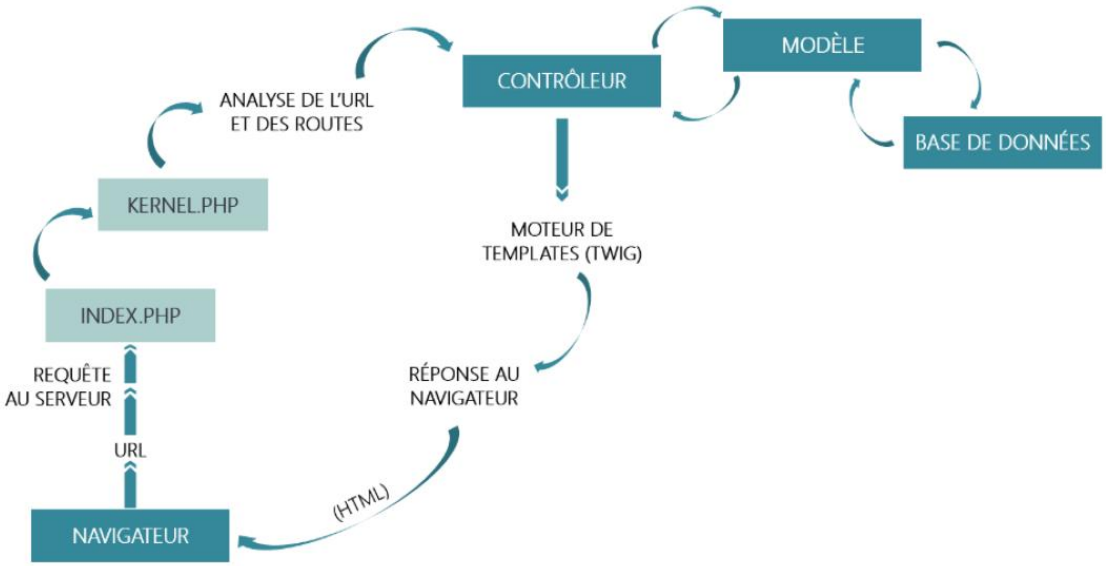
Développement des contrôleurs et gestion de routage

Lors de la création d'applications complexes, nous utilisons un design pattern appelé MVC, pour Modèle-Vue-Contrôleur. Ce modèle impose une séparation de trois couches distinctes : le Modèle, qui regroupe la logique métier et les données, la Vue, qui assure le rendu de l'interface utilisateur, et le Contrôleur. Ce dernier est la couche intermédiaire qui orchestre les échanges entre le Modèle et la Vue. Cette séparation des responsabilités comporte plusieurs bénéfices, car elle permet la maintenabilité et la réutilisabilité du code.

Il faut noter que ce design pattern peut varier d'un framework à l'autre, et dans le cas de Symfony, ce modèle est légèrement plus complexe. La séparation entre les couches est conservée, mais la structure est enrichie par des composants supplémentaires qui structurent rigoureusement l'application.

Le Kernel, ou noyau, organise le traitement des requêtes HTTP, tandis que le conteneur de services centralise et automatise l'injection de dépendances. Dans Symfony, ce design pattern se concrétise par un système de routage dans les contrôleurs. Le routage est un système qui associe une URL à une méthode spécifique du Contrôleur.

Quand on accède à une URL, le routeur identifie le contrôleur correspondant et exécute la méthode associée. Le Contrôleur fait donc appel aux services nécessaires, interagit avec le Modèle pour récupérer les données et retourne une réponse sous forme de Vue rendue en HTML.



Dans le cas de l'application que nous avons développée, les routes devaient être appelées d'une manière spécifique. D'une part, pour assurer une cohérence globale, et d'autre part, parce que certaines fonctionnalités nécessitaient l'utilisation de routes d'API. Ainsi, lorsqu'une requête est envoyée à l'une de ces routes, le contrôleur interroge l'ERP via son API et récupère les données nécessaires à l'application. Cette approche permet de découpler l'ERP du reste de l'application. La route dédiée à l'upload de documents n'utilise pas une route d'API, car elle ne récupère pas de données, mais elle en transmet.

```php
#[IsGranted('ROLE_USER')]
#[Route('user/intervention', name: 'app_')]
final class InterventionController extends AbstractController
{#[Route(
        '/{intervention}/upload-document',
        name: 'intervention_upload_doc',
        methods: ['GET', 'POST']
    )]
    public function upload(
        Intervention $intervention,
        Request $request,
        CreateDocumentUseCase $useCase,
        GetInstructorUseCase $getInstructorUseCase,
    ): Response {
        $currentUser = $this->getUser();
        if (!$currentUser) {
            return $this->redirectToRoute('app_login');
        }
        /** @var User $user */
        $user = $currentUser;
        $instructorId = $getInstructorUseCase->findIdByUserId($user->getId());
        $document = new DocumentModelData();
        $form = $this->createForm(DocumentType::class, $document);
        $form->handleRequest($request);
        if ($form->isSubmitted() && $form->isValid()) {
            try {
                $useCase->create($document, $instructorId, $intervention->getId());
                $this->addFlash('success', "Le document e été ajouté avec succès");
                return $this->redirectToRoute(
                    'app_user_intervention_page',
                    ['intervention' => $intervention->getId(), 'instructor' => $instructorId]
                );
            } catch (CannotCreateDocumentException $e) {
                $this->addFlash('error', $e->getMessage());
            }
        }
        return $this->render(
            'domains/document/uploadDocument.html.twig',
            ['form' => $form->createView(), 'instructorId' => $instructorId, 'intervention' =>
$intervention]
        );
    }
}
```

Comme nous pouvons le constater, la route /user/intervention/{intervention}/upload-document est accessible via les méthodes HTTP GET et POST. La route est protégée par l'attribut isGranted('ROLE_USER'), qui limite les accès uniquement aux utilisateurs ayant le rôle 'USER', donc aux formateurs. La méthode upload reçoit en paramètres l'intervention concernée, la requête HTTP et deux services : CreateDocumentUseCase, qui est l'use case contenant la logique métier relative à l'upload de documents précédemment décrite, et GetInstructorUseCase, un autre service qui se charge de récupérer l'ID du formateur connecté.

Après avoir vérifié que le formateur est authentifié, la méthode upload récupère son identifiant. Ensuite, elle construit le formulaire DocumentType pour récupérer le document et les informations qui lui sont liées. Ce n'est qu'après soumission et validation du formulaire qu'on appelle le service CreateDocumentUseCase pour uploader le document et enregistrer ses informations dans la base de données. Si l'opération réussit, un message de réussite est affiché

et le formateur est redirigé vers la page de détail d'une intervention. En cas d'échec, le formulaire est réaffiché avec les erreurs commises.

```php
#[IsGranted('ROLE_USER')]
#[Route('user/intervention', name: 'app_')]
final class InterventionController extends AbstractController
{
  #[Route('/{intervention}/document/{document}', name: 'intervention_document', methods: ['GET'])]
    public function showOneDocument(
        Intervention $intervention,
        Document $document,
        GetDocumentUseCase $useCase,
    ): Response {
        $interventionID = $intervention->getId();
        $docID = $document->getId();
        $doc = $useCase->findOneDocumentByInterventionId($interventionID, $docID);
        $fileURL = $doc->fileURL;
        $ext = $doc->mimetype;
        $dir = $this->getParameter('kernel.project_dir');
        $url = $dir . '/' . $fileURL;
        $response = new BinaryFileResponse($url);
        $response->headers->set('Content-Type', $doc->mimetype);
        $downloadName = pathinfo($document->getName(), PATHINFO_FILENAME) . '.' . $ext;
        $response->setContentDisposition(
            ResponseHeaderBag::DISPOSITION_ATTACHMENT,
            $downloadName
        );
        return $response;
    }
}
```

Nous avons codé une méthode complémentaire, nommée showOneDocument, qui se charge du téléchargement d'un document. Cette méthode sert à valider l'upload des documents, en confirmant que la fonctionnalité de téléversement a bien fonctionné. Le document, une fois envoyé avec la méthode upload, doit être correctement stocké et associé à l'intervention et au formateur en base de données. La méthode showOneDocument permet de récupérer ce même fichier et de vérifier qu'il est accessible, intègre et correspond bien au document attendu.

Conception et intégration de l’interface utilisateur (IHM) avec twig et Tailwind

La création de l'interface utilisateur est une des étapes les plus essentielles, car elle constitue le point de contact entre l'utilisateur et l'application. Cette interface désigne l'apparence et l'interactivité d'une page web. En tant que développeurs, nous avons le devoir de rendre cette interface la plus ergonomique possible, afin de faciliter et d'agrémenter la navigation pour les utilisateurs.

Puisque l'application appartient à un établissement avec sa propre identité numérique, la conception de l'interface utilisateur devait respecter les codes couleurs de l'entreprise.

Pour cela, nous nous sommes servis de Twig, un moteur de templates pour PHP. Cet outil permet de créer des modèles de pages web et, à travers des syntaxes simples, d'insérer des données dynamiques dans les pages HTML.

```
{% extends 'base.html.twig' %}
{% block body %}
/*ceci n'est qu'un extrait du code*/
                    {% if interventionDetail.review.backlogReviewsDTO is not null %}
                        <div class="bg-slate-50 px-4 py-2 border-t border-slate-200 flex justify-
between">
                            <a href="{{ path('app_user_intervention_backlogReviews',
{'intervention':interventionDetail.id, 'instructor':instructor.id,
'review':interventionDetail.review.id}) }}"
                               class="text-sm text-blue-600 hover:text-blue-800 flex items-center">
                                <svg class="w-4 h-4 mr-1" fill="none" stroke="currentColor" viewBox="0
0 24 24"
                                     xmlns="http://www.w3.org/2000/svg">
                                    <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                          d="M7 16V4m0 0L3 8m4-4l4 4m6 0v12m0 0l4-4m-4 4l-4-4"></path>
                                </svg>
                                Voir les versions antérieures
                            </a>
                            <a href="{{ path('app_user_update_review',{'instructor':instructor.id,
'intervention':intervention.id}) }}" class="text-sm text-blue-600 hover:text-blue-800 flex items-
center">
                                <svg class="w-4 h-4 mr-1" fill="none" stroke="currentColor" viewBox="0
0 24 24"
                                     xmlns="http://www.w3.org/2000/svg">
                                    <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                          d="M11 5H6a2 2 0 00-2 2v11a2 2 0 002 2h11a2 2 0 002-2v-5m-
1.414-9.414a2 2 0 112.828 2.828L11.828 15H9v-2.828l8.586-8.586z"></path>
                                </svg>
                                Modifier le compte rendu
                            </a>
                        </div>
                    {% endif %}
                </div>
            {% endif %}
        </div>
        <div class=" mt-12 mb-8 mx-4 ">
            <div class="bg-ardoise-100 rounded-t-md">
                <div class="flex items-center ">
                    <div class="ml-4">
                        <?xml version="1.0" encoding="UTF-8"?>
                        <svg width="24px" height="24px" viewBox="0 0 24 24" stroke-width="1.5"
fill="none"
                             xmlns="http://www.w3.org/2000/svg" color="#ffffff">
                            <path d="M7 18H10.5H14" stroke="#ffffff" stroke-width="1.5" stroke-
linecap="round"
                        </svg>
                    </div>
                    <h4 class="font-semibold text-white flex items-center p-2">Documents de
l'intervention</h4>
                </div>
            </div>
            {% include 'domains/document/documents.html.twig' with {'documents':
intervention.documents, 'interventionId':intervention.id } %}
        </div>
    </div>
{%endblock%}
```

Dans l'extrait de code, quelques lignes sont soulignées pour illustrer certaines utilisations de Twig. En effet, avec Twig, on peut conditionner l'affichage de certaines parties de la page HTML en fonction des données dynamiques transmises. Dans le deuxième cas, on peut chercher une URL directement par le nom donné à une route définie dans la classe de contrôleurs, en lui passant les identifiants nécessaires à l'appel de la route. Enfin, le dernier exemple illustré dans cet extrait est l'inclusion d'un template Twig dans un autre, afin de favoriser la réutilisabilité du code et d'obtenir des templates plus propres et mieux organisés.

On peut remarquer que l'extrait de code ci-dessus est encombré par la présence de nombreuses classes de style. Cela est dû à l'obligation d'utiliser Tailwind CSS pour la stylisation des pages de l'application. Il s'agit d'un framework CSS dit "utility-first". Contrairement à l'approche traditionnelle qui consiste à écrire le CSS dans un fichier dédié, Tailwind applique les styles directement via les classes HTML.

Cette partie a été une des plus difficiles. La difficulté du front end réside dans le fait qu'il ne suffit pas de savoir coder, mais faut aussi avoir l'œil pour créer des interfaces agréables et intuitives. Le fait que le site ait une identité numérique à rendu les choses significativement plus simples. Développer l'interface utilisateur est un exercice qui requiert à tout développeur de se mettre à la place d'un utilisateur qui ne serait pas à l'aise avec des outils informatiques. Ça aide à remettre en perspective tout le travail de développement. Il ne s'agit plus de coder un programme qui accomplit des tâches mais, de créer un outil qui va être utilisé par différentes personnes pour leur simplifier une tâche.

Stratégie de tests unitaires

La phase de test est l'une des plus importantes du développement car elle confirme ou pas que le travail réalisé jusqu'à présent produit les résultats souhaités.

Nous avons surtout réalisé des tests unitaires pour examiner le bon déroulement d'une fonctionnalité. En testant une petite partie isolée du code, on peut déceler immédiatement des bugs.

Le fait de réaliser des tests unitaires sert aussi lors de la modification d'une fonctionnalité. En effet, la modification du code peut entraîner la casse de celui-ci et le fait de pouvoir vérifier à travers des tests après une modification permet de prévenir cette casse.

```php
<?php

namespace App\Tests;

use App\Domain\Document\Exception\CannotCreateDocumentException;
use App\Domain\Document\Model\DocumentModelData;
use App\Domain\Document\UseCase\CreateDocumentUseCase;
use App\Entity\Instructor;
use App\Entity\Intervention;
use App\Repository\InstructorRepository;
use App\Repository\InterventionRepository;
use App\Service\FileUploaderService;
use Doctrine\ORM\EntityManagerInterface;
use PHPUnit\Framework\MockObject\MockObject;
use PHPUnit\Framework\MockObject\Stub;
use PHPUnit\Framework\TestCase;
use Symfony\Component\HttpFoundation\File\Exception\FileException;
use Symfony\Component\HttpFoundation\File\UploadedFile;
use Throwable;

class CreateDocumentUseCaseTest extends TestCase
{
    private CreateDocumentUseCase $createDocumentUseCase;
    /** @var InstructorRepository&MockObject */
    private InstructorRepository $instructorRepository;
    /** @var InterventionRepository&MockObject */
    private InterventionRepository $interventionRepository;
    /** @var FileUploaderService&MockObject */
    private FileUploaderService $fileUploaderService;
    /** @var EntityManagerInterface&MockObject */
    private EntityManagerInterface $entityManager;

    public function setUp(): void
    {
        $this->interventionRepository = $this->createMock(InterventionRepository::class);
        $this->instructorRepository = $this->createMock(InstructorRepository::class);
        $this->entityManager = $this->createMock(EntityManagerInterface::class);
        $this->fileUploaderService = $this->createMock(FileUploaderService ::class);
        $this->createDocumentUseCase = new CreateDocumentUseCase(
            $this->interventionRepository,
            $this->instructorRepository,
            $this->entityManager,
            $this->fileUploaderService,
        );
    }

    /**
     * @throws Throwable
     * @throws CannotCreateDocumentException
     */
    public function testCreateDocumentSuccess(): void
    {
        $documentData = $this->createMock(DocumentModelData::class);
        $documentData->name = 'PDFtestupload';
        $documentData->file = $this->createMock(UploadedFile::class);
        $intervention = $this->createMock(Intervention::class);
        $intervention->method('getCode')->willReturn('INT-001');
        $instructor = $this->createMock(Instructor::class);

        $this->interventionRepository
            ->expects($this->once())
            ->method('find')
            ->with(1)
            ->willReturn($intervention);

        $this->instructorRepository
            ->expects($this->once())
            ->method('find')
            ->with(1)
            ->willReturn($instructor);

        $this->fileUploaderService->method('fileSize')->willReturn(1024);
        $this->fileUploaderService->method('mimetype')->willReturn('application/pdf');
        $this->fileUploaderService->expects($this->once())->method('upload');
        $this->entityManager->expects($this->once())->method('persist');
        $this->entityManager->expects($this->once())->method('flush');
        $this->createDocumentUseCase->create($documentData, 1, 1);
    }
```

L'objectif de ce test est de valider que la création d'un document se déroule correctement dans différents scénarios, sans avoir recours à une base de données réelle ou à un système de fichiers physique. Pour ce faire, le test utilise des "mocks", c'est-à-dire des objets simulés qui imitent le comportement des dépendances réelles comme les repositories, le service d'upload ou l'EntityManager de Doctrine. Ces mocks permettent de contrôler finement l'environnement d'exécution et de vérifier que les interactions attendues entre les différentes couches de l'application ont bien lieu.

La méthode setUp est exécutée avant chaque test et sert à initialiser l'environnement. Elle crée des mocks pour chaque dépendance du cas d'usage, à savoir le repository des interventions, le repository des formateurs, le service de téléversement de fichiers et l'EntityManager. Ensuite, elle instancie le CreateDocumentUseCase en lui injectant ces mocks. Grâce à cette approche, le test est totalement isolé : il ne dépend ni de l'état de la base de données, ni de la présence effective de fichiers sur le disque, ni du bon fonctionnement des services externes. Chaque test peut ainsi se concentrer uniquement sur la logique métier du cas d'usage.

Dans ce test, on prépare des données simulées : un objet DocumentModelData avec un nom valide et un fichier factice, ainsi qu'une intervention et un formateur factices. On configure ensuite les mocks pour qu'ils se comportent comme prévu : le repository des interventions doit trouver une intervention dont l'identifiant est 1, le repository des formateurs doit trouver un formateur avec le même identifiant, et le service d'upload doit renvoyer des informations sur la taille et le type MIME du fichier. On s'attend également à ce que la méthode upload du service soit appelée une fois, de même que les méthodes persist et flush de l'EntityManager. Enfin, on exécute la méthode create du cas d'usage avec les données préparées. Si tout se passe bien et qu'aucune exception n'est levée, le test est considéré comme réussi, ce qui confirme que le flux de création fonctionne comme prévu.

Un test unitaire ne doit pas se contenter de vérifier les cas où la fonctionnalité marche dans les conditions idéales, mais doit aussi prouver que la fonctionnalité ne marchera pas dans le cas d’utilisation de données erronées.

```php
<?php

namespace App\Tests;

class CreateDocumentUseCaseTest extends TestCase
{
    private CreateDocumentUseCase $createDocumentUseCase;
    /** @var InstructorRepository&MockObject */
    private InstructorRepository $instructorRepository;
    /** @var InterventionRepository&MockObject */
    private InterventionRepository $interventionRepository;
    /** @var FileUploaderService&MockObject */
    private FileUploaderService $fileUploaderService;
    /** @var EntityManagerInterface&MockObject */
    private EntityManagerInterface $entityManager;

    public function setUp(): void
    {}

    /**
     * @throws Throwable
     */
    public function testCreateDocumentFailureInvalidCharacters(): void
    {
        $documentData = $this->createMock(DocumentModelData::class);
        $documentData->name = 'fileName!@#$';

        $this->expectException(CannotCreateDocumentException::class);
        $this->expectExceptionMessage("Seuls les caractères spéciaux suivants sont admis : - et _");
        $this->createDocumentUseCase->create($documentData, 1, 1);
    }
```

Dans ce test on explore le cas d'échec lié à la validation du nom du fichier. Ici, on prépare un objet DocumentModelData dont le nom contient des caractères spéciaux non autorisés, comme des points d'exclamation ou des arobases. Le test indique à PHPUnit qu'il s'attend à recevoir une exception de type CannotCreateDocumentException avec un message d'erreur spécifique indiquant que seuls les caractères spéciaux - et _ sont admis. Lorsque l'on exécute la méthode create, le cas d'usage doit détecter que le nom est invalide et lever cette exception. Si c'est bien le cas, le test réussit, prouvant ainsi que la validation du nom fonctionne correctement et que les erreurs sont remontées de manière appropriée.

```php
<?php

namespace App\Tests;

class CreateDocumentUseCaseTest extends TestCase
{
    private CreateDocumentUseCase $createDocumentUseCase;
    /** @var InstructorRepository&MockObject */
    private InstructorRepository $instructorRepository;
    /** @var InterventionRepository&MockObject */
    private InterventionRepository $interventionRepository;
    /** @var FileUploaderService&MockObject */
    private FileUploaderService $fileUploaderService;
    /** @var EntityManagerInterface&MockObject */
    private EntityManagerInterface $entityManager;

    public function setUp(): void
    {}

    /**
     * @throws Throwable
     */
    public function testCreateDocumentFailureUploadDocument(): void
    {
        $documentData = $this->createMock(DocumentModelData::class);
        $documentData->name = 'AbsentFile';
        $documentData->file = $this->createMock(UploadedFile::class);
        $intervention = $this->createMock(Intervention::class);
        $intervention->method('getCode')->willReturn('INT-001');
        $instructor = $this->createMock(Instructor::class);

        $this->interventionRepository
            ->expects($this->once())
            ->method('find')
            ->with(1)
            ->willReturn($intervention);

        $this->instructorRepository
            ->expects($this->once())
            ->method('find')
            ->with(1)
            ->willReturn($instructor);

        $this->fileUploaderService->expects($this->once())->method('upload')
            ->willthrowException(new FileException("une erreur est survenue lors de l'upload du
fichier."));

        $this->expectException(CannotCreateDocumentException::class);
        $this->createDocumentUseCase->create($documentData, 1, 1);
    }
}
```

Dans ce test on étudie le cas d'un échec lors de la phase d'upload d'un document. Dans ce scénario, on prépare des données similaires au premier test, mais on configure le mock du service d'upload pour qu'il lève une exception de type FileException lorsque la méthode upload est appelée, simulant ainsi une erreur technique lors de l'enregistrement du fichier sur le serveur. Cette exception est capturée par l'use case de création (CreateDocumentUseCase) qui transforme l'exception en une exception métier CannotCreateDocumentException, qui présente l'erreur de manière cohérente avec le domaine de l'application. Si l'exception est levée, le test est validé.

Sécurisation de l'application et veille technologique (authentification, autorisation, protection CSRF, etc.)

Lors du développement de quelconque application, une des préoccupations principales est la sécurité. C'est un fondement de la structure de l'application et il faut l'intégrer dès le début. On pourrait condenser les actions entreprises pour assurer la sécurité d'une application en trois catégories : gestion des accès, protection contre les attaques courantes et gestion des données sensibles

1. *Gestion des accès : authentification et autorisation*

Notre application est une application interne. Par défaut, elle doit être accessible uniquement aux personnes rattachées à l'établissement. C'est pour cette raison qu'on a mis en place un système d'authentification grâce au SecurityBundle de Symfony, qui gère automatiquement la partie authentification. La méthode d'authentification choisie est basée sur l'adresse mail d'un utilisateur et son mot de passe.

On a préféré cette méthode puisqu'elle permet à des intervenants externes à l'établissement de se connecter à l'application. L'autre méthode envisagée était la mise en place d'un OAuth, qui est un système d'authentification permettant à une application secondaire, dans ce cas Microsoft Azure, de se connecter à l'application développée sans avoir à saisir le mot de passe. La raison pour laquelle cette méthode a été abandonnée c'est justement à cause des intervenant extérieurs, qui n'existent pas dans la base de données de Microsoft Azure. Cette connexion OAuth sera mise en place pour une version 2 de cette application, mais uniquement pour les administrateurs, puisqu'ils sont enregistrés dans la base de données de Microsoft Azure de l'établissement.

Pour ce qui concerne les autorisations, elles sont aussi gérées par le SecurityBundle de Symfony. En effet, l'application est conçue pour deux types d'utilisateur : les formateurs et les administrateurs. C'est pour cette raison que le système d'autorisation est nécessaire parce qu'il y a des informations ou des actions que devraient être réservées soit à l'un soit à l'autre. Ainsi, la fonctionnalité d'upload de documents est uniquement réservée aux formateurs (et intervenants externes), c'est pour cela que la route est sécurisée à la racine par l'attribut isGranted('ROLE_USER3') (cf. page 33).

2. *Protection contre les attaques courantes*

Une préoccupation clé lors de la mise en place de la sécurité est la protection contre les attaques courantes.

Il y a deux attaques contre lesquelles nous nous sommes protégés lors du développement de l'application.

La première concerne les injections SQL. Les injections SQL est une attaque au cours de laquelle des commandes SQL malveillantes sont insérées dans des champs de saisie. Ces commandes ont pour but d'affecter l'intégrité de la base de données.

Pour s'en protéger, nous avons utilisé des requêtes paramétrées. Il s'agit de requêtes dans laquelle, les valeurs des variables, donc des données, ne sont pas directement écrites dans la requête mais sont remplacées par des placeholders. Ces derniers sont liés à des valeurs réelles que la base de données traite séparément du code SQL, empêchant toute interprétation malveillante.

Une seconde attaque contre laquelle nous nous sommes protégés est le Cross-Site Request Forgery (CSRF) ou falsification de requête intersite. Il s'agit d'une attaque qui force un utilisateur authentifié à exécuter des actions de façon involontaire, et donc à l'insu de l'utilisateur. Cette attaque concerne surtout les formulaires, et avec Symfony, on en est

protégés par défaut. Symfony attribue un token (chaine de caractères aléatoire unique et imprévisible) au formulaire lors de la création de ce dernier. Le token est stocké en session de l'utilisateur. Lorsque le formulaire est soumis et validé on vérifie que le token en session est le même que celui reçu. S'il y a des token différents, la requête du formulaire est rejetée.

3. Gestion des données sensibles

Dans une application comme la nôtre, les données les plus sensibles sont les mots de passe. C'est pour cette raison qu'on a haché le mot de passe de chaque utilisateur avant de l'insérer dans la base de données.

Le hachage est un processus visant à transformer une donnée, dans ce cas un mot de passe en une chaîne de caractères de taille fixe, irréversible. Le hachage met en place une relation unidirectionnelle d'égalité entre le mot de passe originel et la chaîne de charactères obtenus après hachage.

Pour réaliser le hachage nous nous sommes servis de l'algorithme bcrypt qui est intégré à Symfony.

Pendant le stage, la lecture de documentation et la veille technologique nous a permis de prendre conscience des attaques auxquels on pourrait être confrontés. Pour cela le site owasp a été essentiel. Une autre chose qui nous a permis de rendre plus sécurisée notre application c'est d'utiliser les technologies les plus récentes pour coder notre projet. La raison est simple, les failles de sécurités n'ont pas encore été découvertes. Utiliser la vielle version des technologies ne permet pas à l'application de bénéficier de correctifs de sécurités et de mises à jour pour combattre les attaques. De plus dans les vielles versions, les failles de sécurités sont connues et restent ouvertes.

Conclusion

Au terme de ce stage, l'application est pleinement fonctionnelle et répond à l'ensemble des besoins du client. Les formateurs peuvent consulter et gérer leurs documents et comptes rendus via une interface claire, et toutes les fonctionnalités requises ont été implémentées.

Sur le plan technique, ce stage m'a permis de découvrir des concepts essentiels qui ont enrichi ma pratique du développement. J'ai été initiée à la Clean Architecture, une approche qui structure le code en couches indépendantes et sépare la logique métier des détails techniques.

Bien que déroutante au début, cette architecture rend le code plus modulaire, plus facile à tester et à faire évoluer. J'ai également appris à utiliser systématiquement des DTO plutôt que des entités pour les échanges de données. Cette pratique, qui m'a d'abord déstabilisée car j'étais habituée à la rigidité des entités Doctrine, est devenue un réflexe que j'apprécie pour la sécurité et la flexibilité qu'elle apporte. Enfin, la méthodologie de travail collaborative avec GitLab et la stratégie "une fonctionnalité = une branche" m'ont permis de découvrir une façon professionnelle de développer en équipe, facilitant les revues de code et l'intégration progressive des modifications.

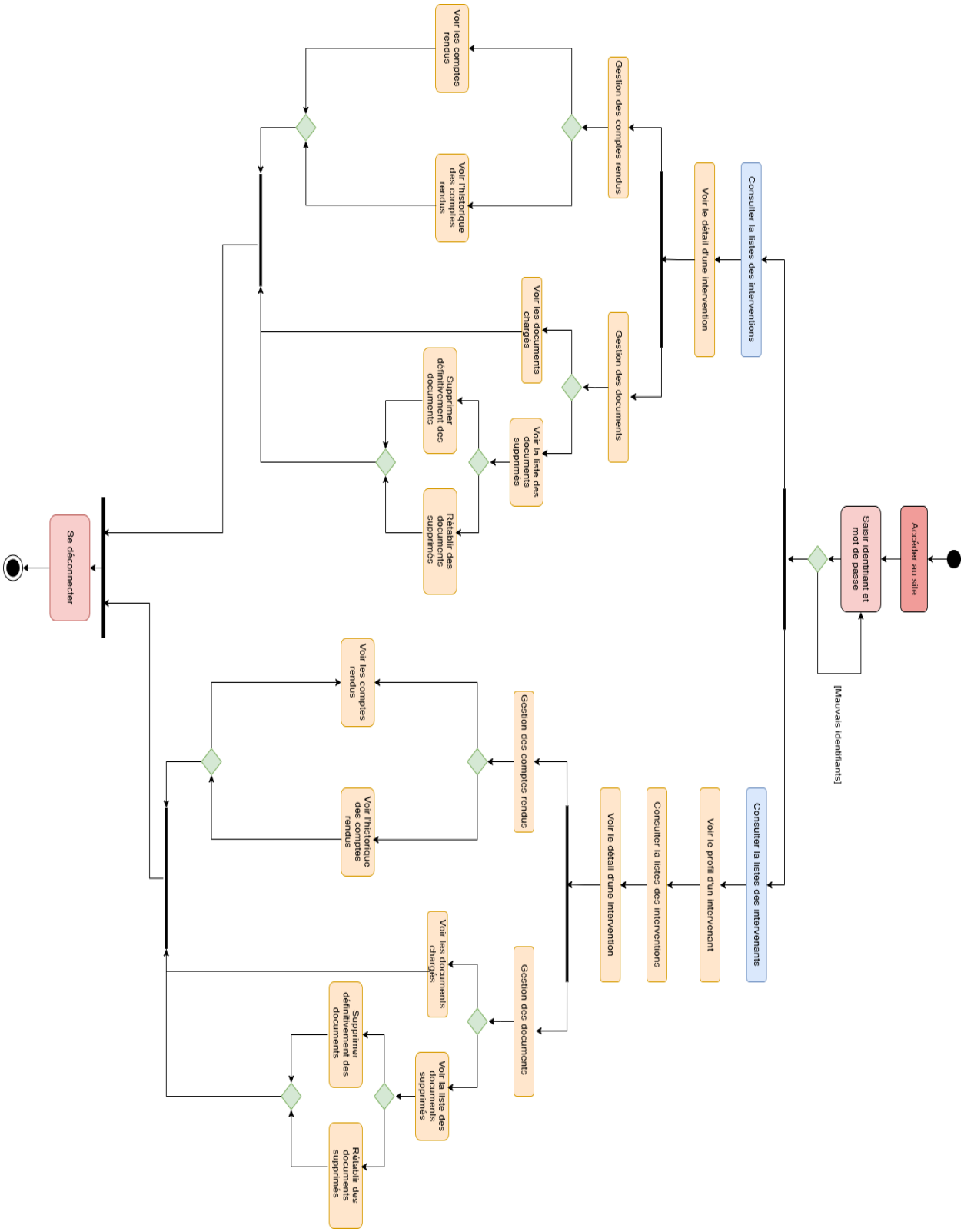
Ce stage m'a permis d'acquérir de nombreuses compétences : maîtrise avancée de Symfony, utilisation des mocks dans les tests unitaires, sensibilisation aux bonnes pratiques de sécurité comme la protection CSRF ou les requêtes paramétrées, le développement en approche DevOps et gestion rigoureuse des versions avec GitLab. J'ai également pris conscience de l'importance de maintenir les dépendances à jour pour garantir sécurité et performances.

Les difficultés rencontrées ont été surmontées grâce à la persévérance et à l'accompagnement de l'équipe. La structuration imposée par la Clean Architecture a nécessité un temps d'adaptation et l'étude attentive de la documentation. Le développement de l'interface utilisateur reste un défi personnel, car je n'ai pas naturellement de sensibilité au design, mais j'ai compensé en privilégiant la simplicité.

En définitive, ce stage a été une expérience extrêmement formatrice. Il m'a permis de passer de la théorie à la pratique, de découvrir de nouvelles approches architecturales et de me confronter aux réalités du développement en entreprise. J'en repars avec des compétences techniques solides, une méthode de travail plus rigoureuse et une vision plus claire de mon futur parcours professionnel.

Annexes

Diagramme d'état



Use case

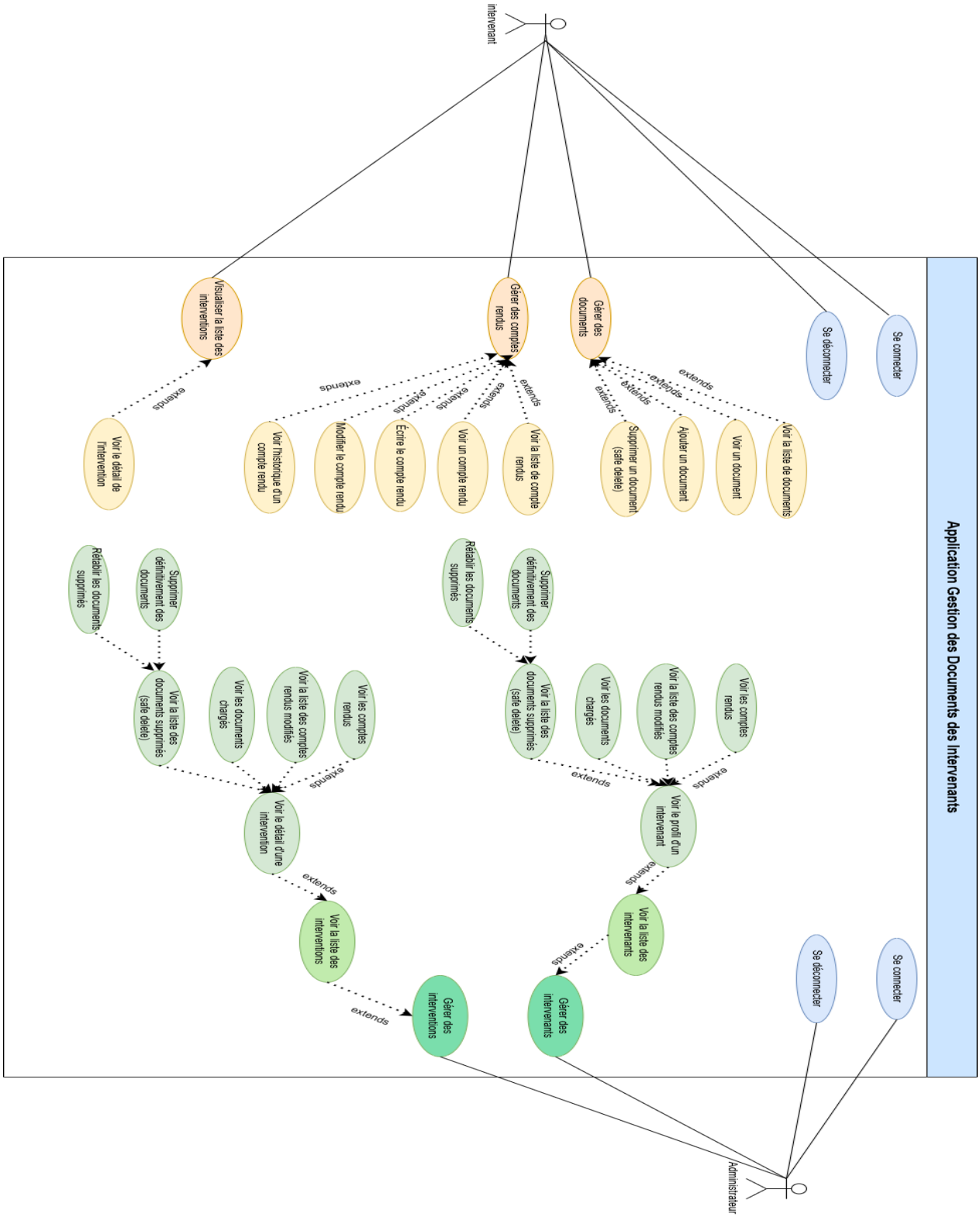
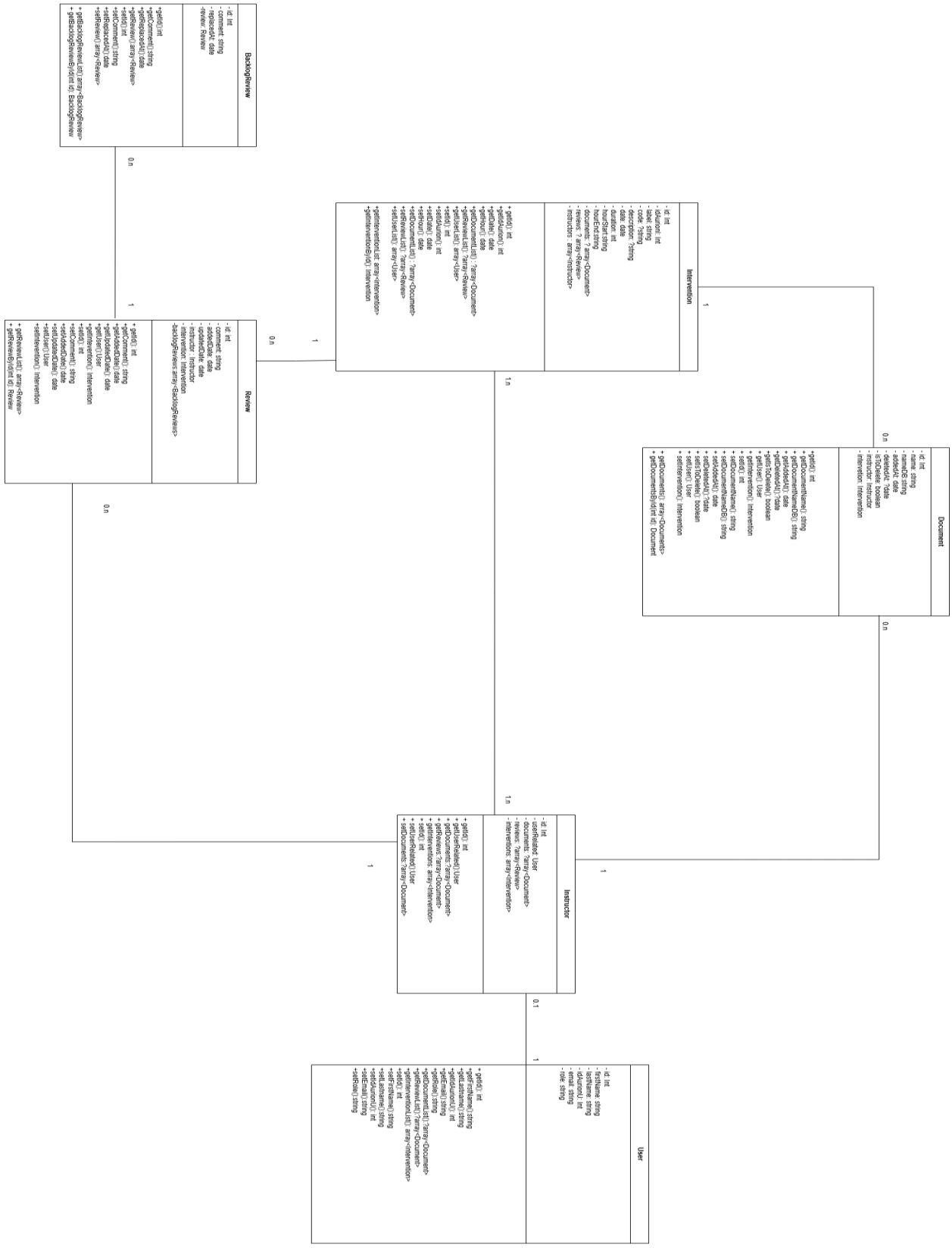
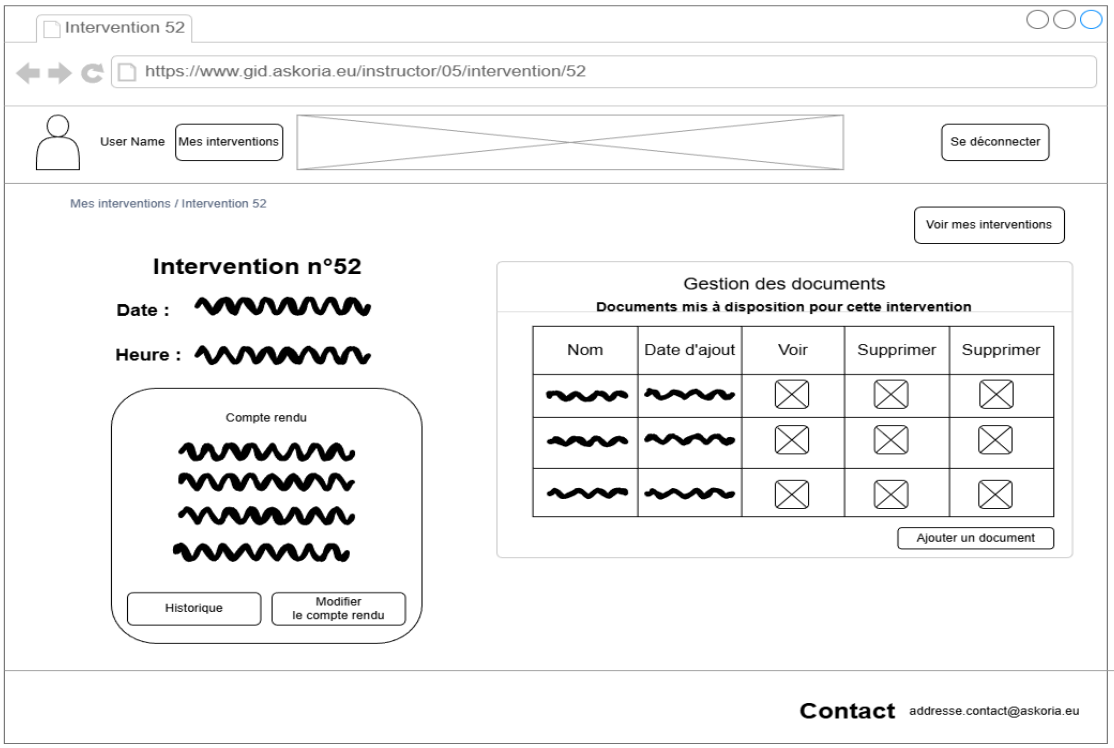
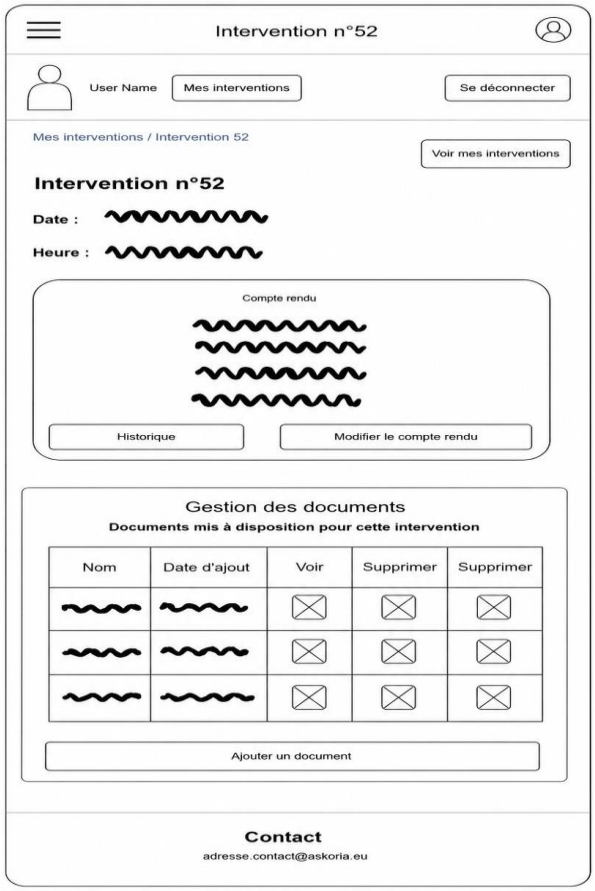

Application Gestion des Documents des Intervenants
Intervenant
Administrateur
Visualiser la liste des interventions
Gérer des comptes rendus
Gérer des documents
Se déconnecter
Se connecter
Voir le détail de l'intervention
Voir l'historique d'un compte rendu
Modifier le compte rendu
Écrire le compte rendu
Voir un compte rendu
Voir la liste de compte rendus
Supprimer un document (safe delete)
Ajouter un document
Voir un document
Voir la liste de documents
Rétablir les documents supprimés
Supprimer définitivement des documents
Voir la liste des documents supprimés (safe delete)
Voir les documents chargés
Voir la liste des comptes rendus modifiés
Voir les comptes rendus
Voir le détail d'une intervention
Voir la liste des interventions
Voir le profil d'un intervenant
Voir la liste des intervenants
Gérer des interventions
Gérer des intervenants
extends

Diagramme de classe



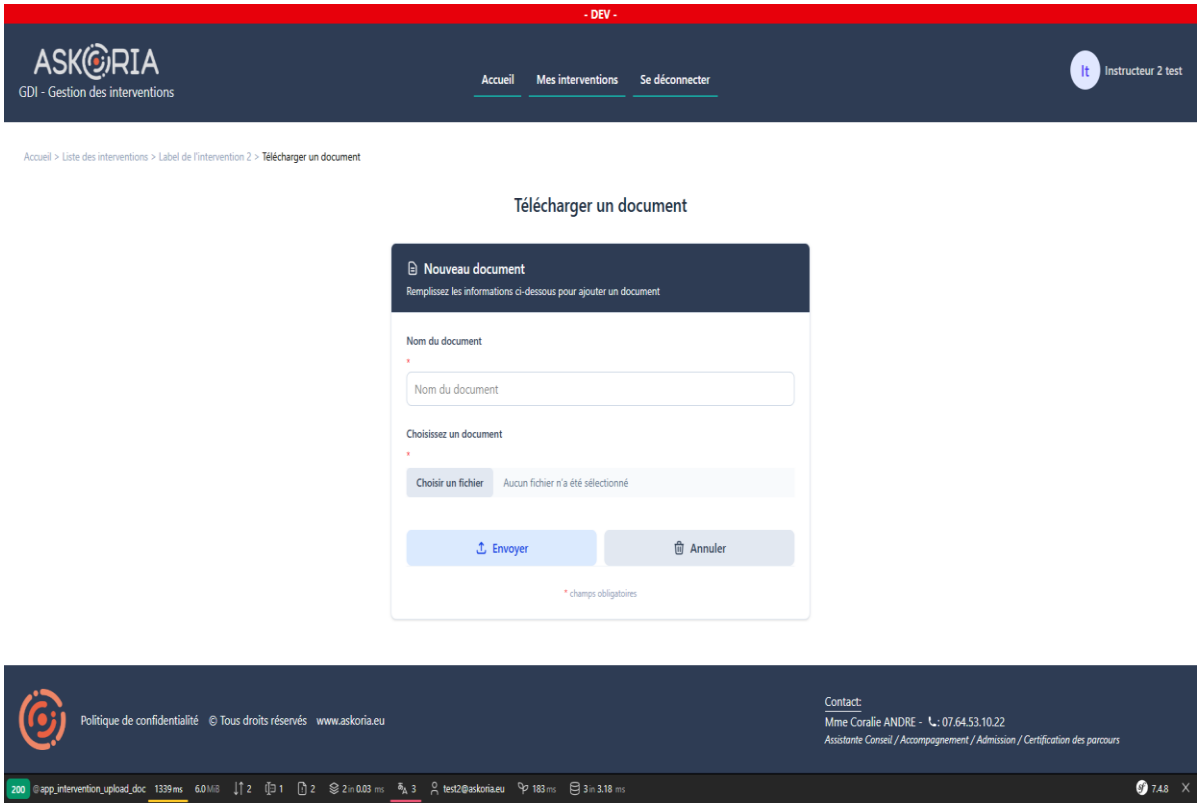
Wireframes pour écran PC et écran mobile


Intervention 52
https://www.gid.askoria.eu/instructor/05/intervention/52
User Name
Mes interventions
Se déconnecter
Mes interventions / Intervention 52
Voir mes interventions
Intervention n°52
Date :
Heure :
Compte rendu
Historique
Modifier le compte rendu
Gestion des documents
Documents mis à disposition pour cette intervention
Nom
Date d'ajout
Voir
Supprimer
Supprimer
Ajouter un document
Contact
addresse.contact@askoria.eu

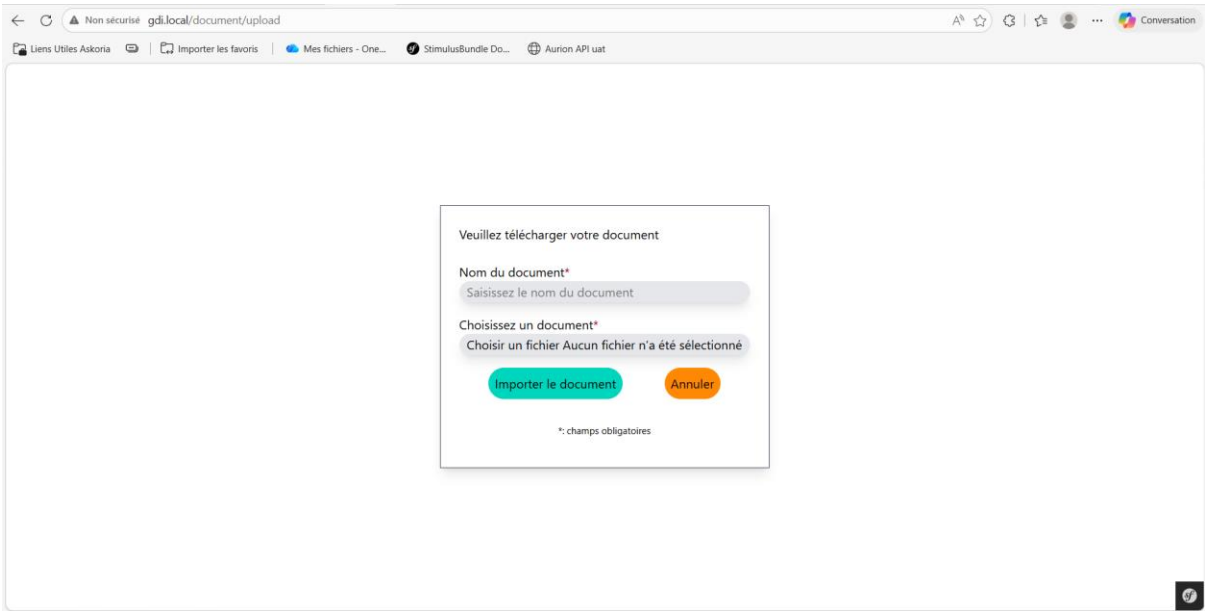

Intervention n°52
User Name
Mes interventions
Se déconnecter
Mes interventions / Intervention 52
Voir mes interventions
Intervention n°52
Date :
Heure :
Compte rendu
Historique
Modifier le compte rendu
Gestion des documents
Documents mis à disposition pour cette intervention
Nom
Date d'ajout
Voir
Supprimer
Supprimer
Ajouter un document
Contact
adresse.contact@askoria.eu

Maquettes

Formulaire d'upload de documents version finale



Formulaire d'upload de document version 1, à l'origine une modale



Nom et prénom du candidat : .....Hailoul Yasmine.....

Document complété d'un commun accord entre le stagiaire et le responsable du stage en entreprise à joindre au rapport d'activité.

Compétences Voir le détail dans le référentiel d'emploi, d'activités et de compétences	Cocher les compétences mises en œuvre lors du projet en entreprise (en totalité ou partiellement)
C1 - Installer et configurer son environnement de travail en fonction du projet web ou web mobile	☒
C2 - Maquetter des interfaces utilisateur web ou web mobile	☒
C3 - Réaliser des interfaces utilisateur statiques web ou web mobile	☒
C4 - Développer la partie dynamique des interfaces utilisateur web ou web mobile	☒
C5 - Mettre en place une base de données relationnelle	☒
C6 - Développer des composants d'accès aux données SQL et NoSQL	☒
C7 - Développer des composants métier coté serveur	☒
C8 - Documenter le déploiement d'une application dynamique web ou web mobile	☒

Observations éventuelles :

Du stagiaire : excellente expérience. Accueil dans l'équipe informatique chalereux et tuteurs attentionnées et très formateurs.

De l'entreprise : .....

Nom et Prénom du stagiaire :
Yasmine Hailoul

Signature :

Entreprise : ASKORIA

Nom du responsable de stage : Patrick Coupé

Signature :